



DEPARTMENT OF ENGINEERING MATHEMATICS

Edge Deployable Rock Detection and Burial Status Classification

Tanishk Nanasaheb Shinde
Megh Kashilkar
Sanskar Sanju Gade
Jitendra Suwalka

A dissertation submitted to the University of Bristol in accordance with the requirements of the degree of Master of Science in the Faculty of Engineering.

11th September, 2026

Supervisor: Dr. Felipe Campelo
Assistant Supervisors: Dr. Ahmad Al-Mallahi
(Dalhousie University / McCain Foods representative) and Jason McLaren

Declaration

This dissertation is submitted to the University of Bristol in accordance with the requirements of the degree of MSc in the Faculty of Engineering. It has not been submitted for any other degree or diploma of any examining body. Except where specifically acknowledged, it is all the work of the Authors.

Tanishk Nanasaheb Shinde
Megh Kashilkar
Sanskar Sanju Gade
Jitendra Suwalka

11th September, 2026

Contents

1	Introduction	1
1.1	Project Context and Motivation	1
1.2	Problem Definition and Edge-Deployment Challenge	1
1.3	Aim, Research Question and Hypothesis	2
1.4	Objectives and Project Contribution	2
1.5	Scope and Boundaries	2
1.6	Dissertation Organisation	2
2	Background and Related Work	3
2.1	Precision Agriculture and Agricultural Stone Detection	3
2.2	Object Detection and Lightweight YOLO Models	3
2.3	Transfer Learning and Data Augmentation	4
2.4	Detection and Counting Metrics	5
2.5	Edge Inference, Numerical Precision and Quantisation	5
2.6	Deployment Runtimes and Inference Benchmarking	6
2.7	Research Gap	6
3	Design and Implementation	8
3.1	Implementation Overview	8
3.2	Dataset Preparation and Detector Selection	8
3.3	YOLO Detection and Output Processing	13
3.4	Edge-Deployment Design and Test Workload	14
3.5	Raspberry Pi Environment and Tooling	14
3.6	Benchmark and Evaluation Protocol	15
3.7	Reliability, Diagnostics and Reproducibility	16
3.8	Chapter Summary	16
4	Critical Evaluation	18
4.1	Evaluation Overview	18
4.2	Detector Selection and Class-Specific Behaviour	20
4.3	Accuracy–Efficiency Trade-offs Across Edge Formats	20
4.4	Resource, Counting and Thermal Behaviour	23
4.5	Operational Stability and Runtime Failures	23
4.6	Deployment Selection	24
4.7	Threats to Validity and Relation to Existing Work	25
4.8	Overall Answer to the Research Question	26
5	Conclusion and Future Work	27
5.1	Overall Conclusion and Answer to the Research Question	27
5.2	Assessment of Project Objectives and Research Hypothesis	27
5.3	Current Project Status and Limitations	28
5.4	Prioritised Future Work	28
5.5	Final Conclusion	28
A	Dataset and Detector Development Evidence	31
B	Raspberry Pi Deployment and Reproducibility	32

C	Extended Raspberry Pi Results	33
D	Stability and Failure Diagnostics	34

List of Figures

3.1	Distribution of the 448 original images across the training, validation and test partitions. The fixed pre-augmentation split separates detector fitting from architecture selection and the later 90-image evaluation workload.	9
3.2	Class-level annotation counts for completely exposed and half-buried rocks in each dataset split. The closely balanced overall totals allow later performance differences to be interpreted primarily as differences in visual difficulty rather than severe label imbalance. . . .	9
3.3	Expansion of the training partition from 313 original images to 2,191 images after six photometric variants were generated per source image. Only training data were augmented; validation and test images remained unchanged.	10
3.4	Dataset-scale summary showing 448 original images, 706 annotated rock objects and two burial-status classes. These figures define the limited-data setting that motivates transfer learning, controlled augmentation and careful held-out evaluation.	11
3.5	Validation and test precision, recall and F1 for YOLOv8n, YOLO11n and YOLO11s. YOLO11s leads several operating-point metrics, showing why the detector choice cannot be justified from a single score in isolation.	12
3.6	mAP50 and mAP50–95 for the three candidate YOLO detectors on validation and test data. The stricter mAP50–95 measure evaluates localisation across multiple IoU thresholds and was the predefined architecture-selection criterion.	12
3.7	YOLO candidates validation quality and efficiency.	13
4.1	Three-image graphical summary of Table 4.1.	19
4.2	Median throughput plotted against mAP50–95 for each deployment format that completed the Raspberry Pi benchmark. Points farther right are faster and points higher on the chart retain stronger localisation quality; ONNX FP32 is omitted because no valid aggregate benchmark was completed.	21
4.3	Median throughput and corresponding mean inference latency for each completed deployment format on Raspberry Pi 5. TFLite INT8 provides the highest speed at 22.62 FPS and the lowest latency at 44.13 ms.	21
4.4	F1, mAP50 and mAP50–95 for the five deployment formats that completed the 90-image benchmark. PyTorch retains the strongest aggregate accuracy, while INT8 trades measurable localisation quality for substantially faster inference.	22
4.5	Mean CPU usage, peak process memory and maximum short-run temperature for each completed format. Faster inference does not translate uniformly into lower CPU load, memory demand or thermal output.	24

List of Tables

3.1	Classes prior to augmentation and partitioning into groups.	8
3.2	Validation comparison of the detectors.	11
3.3	Important Raspberry Pi benchmark environment.	15
3.4	Planned protocol versus actual protocol.	15
4.1	Raspberry Pi benchmark results – in canonical format. FPS is the median throughput. . .	18
5.1	Assessment of the project objectives against completed evidence.	27
A.1	Extended detector-development evidence.	31
C.1	Extended canonical edge metrics which are used in the main evaluation.	33

Abstract

Agricultural rocks can cause field disruption and can cause damage to machinery and equipment, motivating computer vision systems that can identify their location and whether they are buried or not. This dissertation investigates the use of a two-class YOLOv8n detector with the aim of identifying completely exposed and half-buried rocks in agricultural RGB images on a Raspberry Pi 5. The research is about the transition from model development to resource-constrained edge inference rather than intending to cover the broader GPS mapping or automated rock-removal system.

The original dataset was comprised of 448 images and 706 labeled rocks. YOLOv8n was selected from YOLOv8n, YOLO11n and YOLO11s with validation mAP50-95 as the pre-defined primary selection criterion. The chosen detector was then prepared in 6 different representations for deployment: PyTorch FP32, ONNX FP32, TFLite FP32, TFLite INT8, NCNN FP32 and NCNN FP16. The 90-image, 135-rock, 640×640 workload was commonly used to test the detection quality, inference time, throughput, CPU utilisation, process memory, rock-count error, temperature and operating conditions of the CPU hardware of the Raspberry Pi 5.

Five representations resulted in the full benchmark being met. PyTorch FP32 had the best aggregate edge-benchmark detection quality, at mAP50 0.548, but with a higher mean latency of 397.48 ms and only 2.52 median FPS. TFLite INT8 decreased the mean latency to 44.13 ms and increased the median throughput to 22.62 FPS, while the mAP50 dropped to 0.503. NCNN FP16 gave an alternative accuracy-efficiency compromise, with median FPS and mAP50 reaching 13.82 and 0.537, respectively. ONNX FP32 successfully did single image inference but was unable to complete any sustained benchmarking.

Furthermore, TFLite INT8 was able to run around 2,533 inferences in a continuous manner for two minutes. Throttling and reboot did not occur, but temperature rose to about 76–77 °C. Half-buried rocks were still significantly more difficult to recognise than exposed rocks were. This weakness existed prior to edge conversion.

The results support the hypothesis that edge-optimized and reduced-precision representations can greatly enhance the ability of the Raspberry Pi to perform inference efficiently while introducing runtime-dependent accuracy and operational trade-offs. Thus, the TFLite INT8 representation was chosen for the tested CPU-only configuration. The resulting system is an academic edge-deployment proof-of-concept, and not a full-ready agricultural solution.

Supporting Technologies

The principal third-party hardware and software resources used in the project were:

- **Raspberry Pi 5 Model B** — target edge-computing platform for CPU-only deployment and benchmarking of the selected YOLOv8n detector.
- **Official Raspberry Pi 27 W USB-C power supply (5.1 V / 5 A)** — used for the final Raspberry Pi benchmark configuration.
- **Debian GNU/Linux 13** — operating system used on the Raspberry Pi deployment platform.
- **Python 3.13.5** — principal Raspberry Pi programming environment for inference, benchmarking, verification and continuous-inference testing.
- **Ultralytics 8.4.115** — used for YOLO model handling, inference and preparation of alternative deployment representations.
- **PyTorch** — original FP32 representation and reference implementation for the selected detector.
- **ONNX / ONNX Runtime 1.28.0** — used for ONNX FP32 export and CPU inference; isolated inference succeeded but a valid sustained benchmark was not completed.
- **TensorFlow Lite / LiteRT** — used for FP32 and post-training INT8 deployment representations and on-device inference.
- **NCNN** — lightweight inference framework used for FP32 and FP16 deployment on the Raspberry Pi CPU.
- **OpenCV 5.0.0** — image loading, preprocessing and supporting computer-vision operations.
- **NumPy 2.5.1** — numerical and array-based processing in inference and evaluation scripts.
- **Pandas 3.0.5** — organisation and analysis of benchmark outputs and experimental results.
- **psutil 7.2.2** — CPU-utilisation and process-memory monitoring during Raspberry Pi benchmarking.
- **CLIP ViT-B/32** — separate crop-level burial-status classification research branch; it was not part of the verified Raspberry Pi benchmark.
- **SSH and SCP** — remote Raspberry Pi access and transfer of deployment files and benchmark outputs.

Notation and Acronyms

Important acronyms and notation used throughout the dissertation are summarised below. Terms are also defined at first use in the main text.

Acronyms and technical abbreviations

AI	Artificial Intelligence
AP	Average Precision
CLIP	Contrastive Language–Image Pre-training
CPU	Central Processing Unit
F1	Harmonic mean of precision and recall
FP16	16-bit floating-point numerical representation
FP32	32-bit floating-point numerical representation
FPS	Frames Per Second
GPS	Global Positioning System
INT8	8-bit integer numerical representation
IoU	Intersection over Union
MAE	Mean Absolute Error
mAP	mean Average Precision
mAP50	mean Average Precision evaluated at IoU threshold 0.50
mAP50-95	mean Average Precision averaged across IoU thresholds from 0.50 to 0.95 in increments of 0.05
NCNN	Lightweight neural-network inference framework for mobile and embedded deployment
ONNX	Open Neural Network Exchange
RAM	Random Access Memory
RGB	Red–Green–Blue image representation
RSS	Resident Set Size; process memory resident in physical RAM
SCP	Secure Copy Protocol
SPPF	Spatial Pyramid Pooling — Fast
SSH	Secure Shell
TFLite	TensorFlow Lite
LiteRT	Google’s on-device machine-learning runtime ecosystem used for TensorFlow Lite model execution
YOLO	You Only Look Once

Mathematical notation

B_{pred}	Predicted bounding box
B_{gt}	Ground-truth bounding box
TP	Number of true-positive detections
FP	Number of false-positive detections
FN	Number of false-negative detections
y_i	Ground-truth rock count for image i
$\hat{y}_i$	Predicted rock count for image i
N	Number of images included in an evaluation
W, H	Width and height of the original image
W', H'	Width and height after aspect-ratio-preserving resizing
s	Letterbox image-scaling factor
r	Real-valued quantity before quantisation
q	Quantised integer representation
S	Quantisation scale factor
Z	Quantisation zero point

Acknowledgements

The authors would like to thank Dr. Felipe Campelo for his academic supervision, guidance and feedback throughout the project. His support helped the group narrow the research question, keep completed evidence and planned work separate, and present the results of the detector-development and edge-deployment with an appropriate level of critical analysis.

The authors would also like to thank the Assistant Supervisors, Dr. Ahmad Al-Mallahi and Jason McLaren, for helping with the overall agricultural-engineering problem. It is worthy to note that the motivation for this project came from the Dalhousie University / McCain Foods project context. This involves investigating field-rock detection, classification of burial status and the constraints that arise if a computer-vision model must operate close to the data collection site.

The group recognises the University of Bristol Department of Engineering Mathematics and the MSc Data Science course for the academic environment, the computing facilities and supporting resources used during the project. Last but not least, the authors acknowledge the teamwork of all four members of the group in dataset preparation, model development, conversion, Raspberry Pi benchmarking, evidence checking, analysis and report preparation. It was necessary to open sharing of experimental outputs and careful crosschecking of outcomes in order to compile a unified collective submission.

Chapter 1

Introduction

1.1 Project Context and Motivation

The results of this group project of four people are presented in this dissertation. The project was undertaken by Tanishk Nanasaheb Shinde, Megh Kashilkar, Sanskar Sanju Gade and Jitendra Suwalka. Unless explicitly stated otherwise, the methods, results and conclusions presented in Chapters 1–5 describe the collective project work rather than the individual group members’ contributions.

Sensing, computer vision and local computation are becoming a trend in precision agriculture to give more timely and spatially precise information for the facilitation of field operations. The importance of object detection is particularly noteworthy because of its ability to transform an image into structured information regarding the location, type and quantity of objects in a scene. Reviews of precision-farming vision systems clearly show the rapid adoption of object detection and the ongoing importance of quality data, variations in the environment, and deployment constraints [1, 2].

The overall topic for this project is “rocks in potato fields”. Rocks can obstruct field operations, contribute to equipment wear and create significant manual removal tasks. The Dalhousie University / McCain Foods project has a vision of detecting rocks during normal field operations, associating detections with position information and finally supporting automated removal [4, 3]. The team project addresses a smaller, verifiable and specific element of that larger vision: detecting rocks in RGB images, distinguishing completely exposed rocks from half-buried rocks, and determining whether the detector can run efficiently on cheap edge devices.

1.2 Problem Definition and Edge-Deployment Challenge

The data available is 448 RGB images and 706 labeled rock instances. The two target classes are *rock completely exposed* and *rock half buried*. This is not just a localisation task. The burial status is important because, if the rock is partially visible, there is less visual evidence, its colour and texture may be similar to the surrounding soil, and it may be more influenced by occlusion. Other literature related to vision of agriculture also refers to background complexity, changing illumination, occlusion and small labelled datasets as common challenges. These were required for successful detection in the field. [1, 22].

It is not always possible to use a detector that works in the development process as a detector on an embedded device. The deployment representation affects the runtime, numerical precision, memory behaviour and supported optimisations, although the underlying trained model itself remains nominally the same. The Raspberry Pi 5 is the newest version of the RP2040, which can be configured in a number of ways. Therefore, the Raspberry Pi 5 gives rise to a system-level engineering question: how accurate is it when? How much latency does each configuration reduce, what are the resources that are used in each runtime, and is each configuration able to run. Does it reliably infer repeatedly?

The project compared three lightweight candidate detectors, YOLOv8n, YOLO11n and YOLO11s, and selected YOLOv8n using validation mAP50–95 as the predefined primary selection criterion. The selected model was then exported to 6 different edge formats: PyTorch FP32, ONNX FP32, TFLite FP32, TFLite INT8, NCNN FP32 and NCNN FP16. The study then distinguishes two decisions which can otherwise be merged: Selecting detector architecture based on *validation results*, and runtime representation Selection according Raspberry Pi behaviour.

1.3 Aim, Research Question and Hypothesis

The main purpose is to explore the potential and the practical considerations of implementing a two-class YOLOv8n agricultural rock detector on a Raspberry Pi 5 using alternative edge-runtime representations. The main research question is:

How effectively can a two-class YOLOv8n rock detector be deployed on a Raspberry Pi 5, and what accuracy, latency, memory and operational trade-offs arise across alternative edge-runtime representations?

The research hypothesis is that reduced-precision and edge-optimised representations will substantially reduce Raspberry Pi inference latency relative to PyTorch FP32, but the amount of improvement will depend on the runtime and could result in a loss of detection accuracy or operational robustness.

1.4 Objectives and Project Contribution

The goals of the project are to:

1. set up the two-class detector and evaluation environment;
2. develop six different edge-runtime representations of the selected detector;
3. create a reproducible evaluation process for Raspberry Pi;
4. benchmark detection quality, latency, throughput and resource usage under a typical workload;
5. explore the reliability of operations and record times when operations did not run successfully; and
6. choose and explain a preferred deployment representation for the tested Raspberry Pi configuration.

The principal contribution, however, is not a new detection architecture. It is an evidence-based transfer of a trained agricultural detector from model-development conditions to constrained hardware. The runtimes were compared on the same images and using a common set of accuracy, speed, counting, resource and operational measures

1.5 Scope and Boundaries

The scope applied covers: dataset auditing, training only augmentation and light YOLO comparison. It also involves choosing the YOLOv8n model, converting it, and performing inference on the stored image using the Raspberry Pi 5, rock counting from detections and cross-runtime benchmarking. A stand-alone CLIP ViT-B/32 branch for crop-classification was also investigated during model development, but it was not part of the verified Raspberry Pi benchmark [14]. The study does NOT imply live-camera operation, integration of GPS, estimating the size or weight of rocks, depth or thermal sensing, robotic manipulation, or implementation of the entire field system. The evaluation focus for the Raspberry Pi is therefore an academic CPU-only proof of concept. It is important because this boundary means that the conclusions relate to the viability of one detection unit and not of an autonomous agricultural machine.

1.6 Dissertation Organisation

Chapter 2 discusses the technical and agricultural background. Chapter 3 describes the executed process from preparing the datasets and detector selection to Raspberry Pi conversion and benchmarking. Chapter 4 critically discusses model generalisation, class-specific weaknesses, accuracy–efficiency trade-offs, resource behaviour and runtime stability. Chapter 5 answers the research question, evaluates the hypothesis and goals, and prioritises the work needed to progress from the current proof of concept to a more realistic field system.

Chapter 2

Background and Related Work

2.1 Precision Agriculture and Agricultural Stone Detection

Precision Agriculture (PA) relies on sensing, computation and decision making based on data at a spatial and temporal resolution smaller than that traditionally used for whole-field management. One way to achieve this goal is through computer vision, which leverages the relatively low cost of camera sensors to provide detailed information about crops, weeds, obstacles and field conditions. More than 100 recent precision-farming studies are reviewed, and the task of detecting and tracking objects is found to be an important enabling technique. Another constant challenge cited is the variability of the environment and the limited amount of labelled data [1]. A literature review of the particular application of YOLO in agriculture also suggests the need for one-stage detectors, as tasks require a practical solution that is accurate and fast [2].

Although significant developments have been achieved by using general-purpose vision models, agricultural-field imagery remains difficult, such as in stone detection. Complex backgrounds are also created by colour, clods, vegetation, shadows and uneven lighting of the soil; a partly buried stone may reveal just a small or irregular area. Thürkow et al. exhibit ongoing research activity in agricultural stone detection via thermal imagery, showing that the problem remains challenging and has thus driven the need for other sensing approaches beyond conventional RGB [22]. The present study uses RGB imagery due to its similarity to that provided in the project, and the low cost vision setting; also, the latter class-specific analysis warrants the importance of the burial remains as a source of error.

2.2 Object Detection and Lightweight YOLO Models

Object detection consists of object classification and object localisation. Two-stage detectors include the steps of proposing regions and classifying/refining them, while one-stage detectors directly predict object categories and locations [19]. In [12], SSD showed that multi-scale detection can be achieved with just one network without the need of a proposal stage. In the one-stage approach, the detection problem was extended to a single unified prediction problem with YOLO, which has become one of the most popular families of real-time detectors [18].

The features used in modern detectors are multi-scale features because agricultural objects can occupy small or large percentages of the image. A previously popular method for integration of semantic information over scales was standardised by Feature Pyramid Networks [10]. Additionally, dense detection was supported by the improvement of bounding-box representation and optimisation, like distribution-based regression in Generalized Focal Loss [9]. While different generations of YOLO do have a number of differences in implementation, these ideas are used to design a small detector which preserves relevant multi-scale information without requiring computation of a large backbone. The actual implementation of YOLOv8/YOLO11 used in this project is from Ultralytics and should be considered alongside the official implementation document, not as a new architecture that we propose.

The project compared 3 models – YOLOv8n, YOLO11n and YOLO11s. The “n” variants are aimed at low parameter and computation budgets, and YOLO11s has greater capacity. The comparison is empirical, meaning that a validation criterion had been predetermined, rather than one generation being believed to be superior to the others.

The Ultralytics detectors can be divided into the backbone, the neck (which performs feature fusion) and the detection head. The backbone learns higher-level feature maps from the pixels, the neck performs

information fusion at different resolutions, and the head performs class and bounding-box predictions. It is important to do multi-scale fusion because a rock may occupy a large portion of one image and a small portion of another image. YOLOv8 is based on C2f blocks and an SPPF module prior to multi-scale fusion, and has an anchor-free decoupled detection head. The overall lightweight single-stage architecture and internal modules such as C3k2/C2PSA are preserved in the official implementation of YOLO11 [23]. The information is relevant to model capacity and computational needs, which are not altered by this project; the experimental question is which of the provided lightweight models generalised best with the same training protocol.

YOLO takes in an image and outputs a list of candidate detections in one inference operation (forward pass). The backbone is employed to derive features from the image, while the neck is responsible for propagating and fusing information across scales and the decoupled head is responsible for bounding-box geometry and class confidence prediction. Those candidates which make it through the filter have a confidence value higher than a certain threshold; overlapping boxes are suppressed during post processing. This sequence returns the box coordinates, class and confidence value for use in annotation, class-specific counting and structured output.

It is important to take into account both localisation and classification in the current task. A partially covered rock has less exposed form and texture than a fully exposed rock, and the detector should be able to register sufficient information to locate the rock, and then be able to classify it as the appropriate class of burial. Therefore, a failure to detect, a misboxed and a mixed burial status are not the same failure, although they impact the same overall score. These behaviors are indicated by reporting precision, recall, F1, mAP50 and mAP50-95 and not by reporting the binary decision, “rock detected”.

The project is built on YOLO and not only is this a convenient pre-processing tool, but it is the primary analytical component. This trained detector is transported from development to every edge-runtime representation, thus enabling the dissertation to differentiate between the weaknesses of the model and the differences caused by the export format, numerical precision or backend execution. By using the controlled continuity, it is possible to make the trade-off between the deployment of the detector and the runtime of the detector.

For edge deployment, the difference between “nano” and “small” applies as well. As model capacity grows, models have more power to represent, but also need more effort per image and may not generalise better on a smaller, but more specialised, set of images. Comparing YOLO11n vs YOLO11s also gives a sense of scale, whilst comparing YOLOv8n vs YOLO11n gives an insight into architecture generation at the edge oriented scale. The selection rule also guarantees that no selection is based only on the age of the model or number of parameters.

2.3 Transfer Learning and Data Augmentation

It is particularly convenient to train from a pretrained representation when the task-specific data are limited. Transfer learning exploits features learned from a larger source set of data and leverages them for a target problem: Yosinski et al. demonstrate that features are not all equally transferable and that they can significantly reduce the time required to learn from scratch [24]. Another common use of large datasets like COCO is as a pretraining and evaluation context for modern detectors [11].

The size of the original project data set is small, which means there is a high risk of overfitting and narrow coverage. Only the training partition was thus extended offline. The following six photometric transformations were applied: brightness, contrast, Gaussian noise, mild blur, hue/saturation and sharpness. In order to increase the number of original training images from 313 to 2,191 training images, these transformations were applied. They generated 1,878 augmented images and 3,563 repeated annotation instances. Importantly, 3,563 rocks were not collected; rather, these instances were augmented copies of previously collected, labelled rocks. The validation and test partitions were not supplemented, to offer a more meaningful estimate of generalisation.

Another important reason why the burial problem cannot be solved entirely by augmentation is the complexity of agricultural scenes. A half-buried rock can be half covered with soil and its boundaries can be indistinct, so the detector needs to use less obvious information than in the case of a fully exposed rock to determine class. This is a semantic and data-distribution issue, rather than just a numerical-precision issue created by edge conversion.

Field imagery is an amalgamation of a variety of factors that are hard to replicate exhaustively with augmentation. Illumination and wet/dry soil reflectance can affect colour and contrast; crop residues and emerging vegetation can create new textures; shadows can create rock-like outlines; and varying camera height/angle of shot can alter the apparent dimensions of an object. Burial is a special case for occlusion,

as the visible fraction is part of the class definition. A detector can make a logical deduction about an object being a rock while still confusing the exposed and half-buried classes. This supports the reporting of mAP and class-specific behaviours rather than a single-digit performance score.

The class totals are fairly evenly distributed overall (362 exposed, 344 half buried), but numerical balance is not visual balance. There may be fewer discriminative pixels in half-buried instances and more ambiguous boundaries. Therefore, future gathering of the data should be done with stratification by class count and by the percentage of burial, as well as lighting, soil type, scene density and rock appearance. This is a better way than merely making more modified versions of the current scenes.

2.4 Detection and Counting Metrics

Intersection over Union (IoU) is a measure of how similar the predicted box B_{pred} is to the ground-truth box B_{gt} .

$$\text{IoU} = \frac{|B_{\text{pred}} \cap B_{\text{gt}}|}{|B_{\text{pred}} \cup B_{\text{gt}}|}. \quad (2.1)$$

A prediction is only deemed a correct detection if its class and overlap satisfy the selected evaluation rules. The metrics used are precision, recall and F1.

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}, \quad F1 = \frac{2PR}{P + R}. \quad (2.2)$$

Average Precision relates the precision–recall relationship at the class level. mAP50 is the average of AP across all classes at IoU 0.50, and mAP50–95 is the average of AP across IoU thresholds from 0.50 to 0.95 with a step size of 0.05. The latter is stricter, as it increasingly rewards boxes localised in the right position.

The project also includes the evaluation of the number of rocks, since in the end a field workflow needs the number of rocks as well as their boxes. The MAE of the N images is

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |\hat{y}_i - y_i|, \quad (2.3)$$

where y_i and $\hat{y}_i$ are the ground-truth rock count and predicted rock count for the i th sample, respectively. Count MAE is provided as an accompaniment to mAP, not as an alternative: a detector can provide a comparable count but assign an incorrect burial-status class or mislocalise the object.

Further, how a measurement is interpreted is also a function of the operating point. The confidence threshold controls the trade-off between recall and false positives, while the overlapping candidate boxes that are retained are controlled by the IoU setting and non-maximum suppression. So, a confusion matrix or count error is not an intrinsic property of a model independent of the choice of threshold. The benchmark sets these parameters and ensures that representations are always compared using the same parameters, with class-specific confusion results used to explain the results rather than replace threshold-integrated AP measures.

2.5 Edge Inference, Numerical Precision and Quantisation

Edge computing allows processing to be done closer to the sensor rather than relying on remote cloud inference. This can help lessen communication latency and dependence on field connectivity in agriculture. Lower latency and local processing are cited as some of the motivations for the combination of cloud, fog and edge technologies in smart agriculture by Kalyani and Collier [8]. Recent reviews emphasise the potential for local intelligent machinery and the importance of increased measurement of accuracy, latency, memory and resource trade-offs [20]. Raspberry Pi systems are already used in a variety of precision-agriculture applications, not only as an affordable experimental platform but also as a platform for conducting deployment research [7].

Edge behaviour is very model-representation dependent. FP32 is a baseline floating point standard. FP16 not only requires less memory and bandwidth, but can also be faster if the target runtime/processor can take advantage of half-precision efficiently. INT8 post-training quantisation maps real values to a range of integers that is limited. An affine form is:

$$q = \text{round}\left(\frac{r}{S}\right) + Z, \quad (2.4)$$

In which r is a real value, q is a quantised integer, S is a scale factor and Z is a zero point. Although an efficient inference of integers can reduce a large amount of computing and memory usage, quantisation can introduce approximation error, as pointed out in [6]. When estimating ranges after training, representative calibration data are important.

Post training quantisation is attractive because the calibration set is embraced into the deployment process, but there is no need to retrain the detector. Difficult examples can be clipped or scaled too coarsely if the representative ranges are poorly estimated. Thus, only original training images were used in the project for INT8 calibration, and there was a balance between the number of calibration objects for the two classes. This does not guarantee lossless quantisation; it does not make the contentious methodological error of tuning it directly to the actual test pictures.

FP16 has another cost-benefit ratio. It saves storage space and numerical bandwidth and is not constrained to an 8-bit integer grid, making it usually easier to get close to FP32 precision. But a theoretical benefit of half precision only comes true in terms of latency if the hardware and runtime make good use of half precision. The latter NCNN result – which was approximately half the model storage but provided only a marginal latency benefit over NCNN FP32 – highlights the need to evaluate the numerical format and realised device speed as separate metrics.

2.6 Deployment Runtimes and Inference Benchmarking

The six representations are used across four software ecosystems in this study. PyTorch FP32 is the original reference representation. ONNX is a portable graph format which is executed here with ONNX Runtime [13]. TFLite/LiteRT offers deployment-ready FP32 and INT8 execution [5]. NCNN is a lightweight inference framework for mobile/embedded CPU environments, which provides support for FP32 and FP16 representations [21].

If considering runtime effects, the workload should be controlled to be fair. The comparison rule for inferences is formalised in MLPerf Inference in which clear scenarios, metrics and reproducible conditions are used to compare. This dissertation is not based on the MLPerf suite, but rather has a similar design: finished representations (from the same stage of processing) will be applied to the same 90 images, with the same input image size of 640×640 and batch size of 1 and the same input parameters for image detection, threshold. All of the above factors are taken into account for detection quality, latency, throughput, CPU utilisation, process memory, temperature and count error.

Latency and FPS don't cut it when it comes to a limited device. A backend can be fast inferring and use a large amount of CPU, but can heat up or crash if the inferences are repeated multiple times. What might be a very accurate representation might not be suitable for an on-the-go workflow. The primary analytical concept of this thesis is therefore the trade-off between accuracy, efficiency and robustness instead of a single leaderboard metric.

A controlled benchmark should also have a clear timing boundary. It is possible to measure end-to-end camera latency, image pre-processing, model execution and post-processing, but it would not be a proper comparison to use different boundaries for the different runtimes. The present experiment assumes that the input geometry is known before the actual experiment and deals specifically with the implemented inference path, while noting that in the future a live-camera system will need to add acquisition and pipeline costs. The first call might contain graph initialisation, memory allocation or cache effects which are not representative of steady execution, hence the warm-up inference.

The mean latency is the average time, and the median FPS is a more representative measure of normal throughput. Resource measures give supplementary information: process RSS – memory resident for the running process; CPU utilisation – how aggressively the backend uses the processor; and temperature – a proxy for the accumulated thermal load. None is sufficient in itself, so the later selection takes account of the evidence put together.

2.7 Research Gap

There are three related points in the literature. Object detection is useful in agriculture but can be a challenge in the field as well [1, 2]. Secondly, edge processing is attractive when the need for latency and/or connectivity is significant; however, deployment studies require evaluating multidimensional resources [8, 20]. Thirdly, Raspberry Pi platforms are powerful enough to allow the implementation of precision-agriculture applications, but are simultaneously constrained systems [7, 16].

YOLO’s ability to detect agricultural objects is then not the issue addressed by the gap. It is the lack of understanding that a detector chosen to operate at development time has a desirable balance of accuracy, latency, and memory usage and runtime characteristics when deployed to alternative runtime representations (such as on lean hardware). If the detector, images and target device do not change, and only the representation changes, then this is a practical choice that can be isolated.

This also sets the dissertation apart from studies that report only on compressed model size or inference speed on a desktop. A format may be acceptable in a logical sense but might not work for a number of reasons, including memory pressure, instability at the device level or software incompatibility; a smaller numerical representation may offer little speedup if the target CPU is not able to gain any benefit from the format. On the other hand, a runtime can deliver significant speed improvements with very little change to the learned weights. It is therefore essential to measure all the formats on the Raspberry Pi to obtain device-specific evidence for expectations about quantisation and lightweight inference.

Then there are two related assessments that need to be performed in the review. The first question is the ability of the chosen detector to be able to generalise from validation to new images of the project and whether both burial classes can be identified similarly. The second one is what portion of this detection behavior persists after deployment, and the speed/cost of each representation. Chapters 3 and 4 maintain this separation to prevent an architecture fault from being misinterpreted as a runtime fault, and vice versa.

The other background issue is the difference between model size, process memory and computing costs. A smaller model file will result in lower storage and transfer requirements, but not a proportionately lower resident memory requirement since this also includes runtime libraries and working buffers. Likewise, a smaller number of bytes per parameter does not necessarily mean lower latency. FLOP-based complexity estimates aid in comparing architectures, but do not account for implementation of operators, scheduling of threads or memory access on a particular CPU. The file/model representation is thus considered to be one factor, and measured RSS, CPU utilisation and latency are used for making a deployment decision.

There should also be negative outcomes that can be seen for reproducibility. If a run-time resets the device during a longer workload, excluding it without explanation would create a bias in favour of successful formats. Likewise, if a couple of pictures are given and a couple of estimated values are added in the table would be complete but not valid. The methodological stance is therefore conservative, and the only metrics provided are for completed common-workload runs; only runs that are incomplete or failed are reported qualitatively, and their evidential limits are identified. In the case of embedded systems, this is especially critical, because software compatibility and operational stability are deployment performance characteristics.

Chapter 3

Design and Implementation

3.1 Implementation Overview

This implementation was made up of 2 aspects, one of which was dependent on the other. The first one was a 2-class detector for the detection of rocks in agricultural RGB images and discrimination between fully exposed and half-buried rocks. The second ported the selected detector to the Raspberry Pi 5 and tested it with alternative representations under a controlled workload of stored images.

3.2 Dataset Preparation and Detector Selection

There are 448 images and 706 annotated rocks in the original dataset. Prior to augmentation, it was partitioned into training data, validation data and test data as displayed in Table 3.1. The test set was comprised of 90 images and 135 labelled rocks and it did not influence the architecture of the detector used. Below is depicted the split of the original 448 images into training, validation and test sets. The pre-

Table 3.1: Classes prior to augmentation and partitioning into groups.

Split	Images	Completely exposed	Half buried
Training	313	259	250
Validation	45	34	28
Test	90	69	66
Total	448	362	344

augmentation split separates detector fitting from architecture selection and the later 90-image evaluation workload. The number of annotations at the class level for completely exposed and half-buried rocks is shown in this chart. The overall totals and distribution of classes are not too extreme, so large label imbalance that could obscure later analyses of the score differences is unlikely.

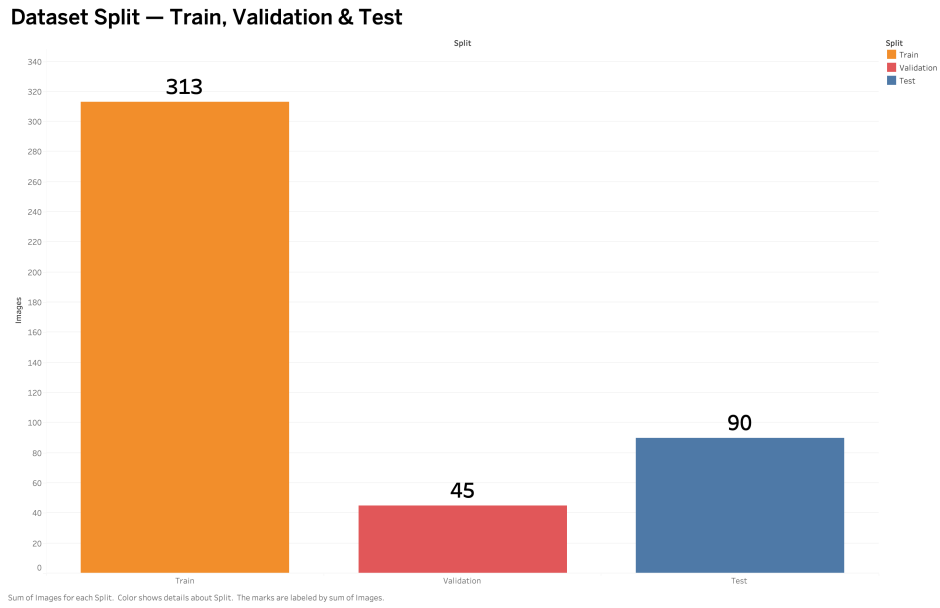


Figure 3.1: Distribution of the 448 original images across the training, validation and test partitions. The fixed pre-augmentation split separates detector fitting from architecture selection and the later 90-image evaluation workload.

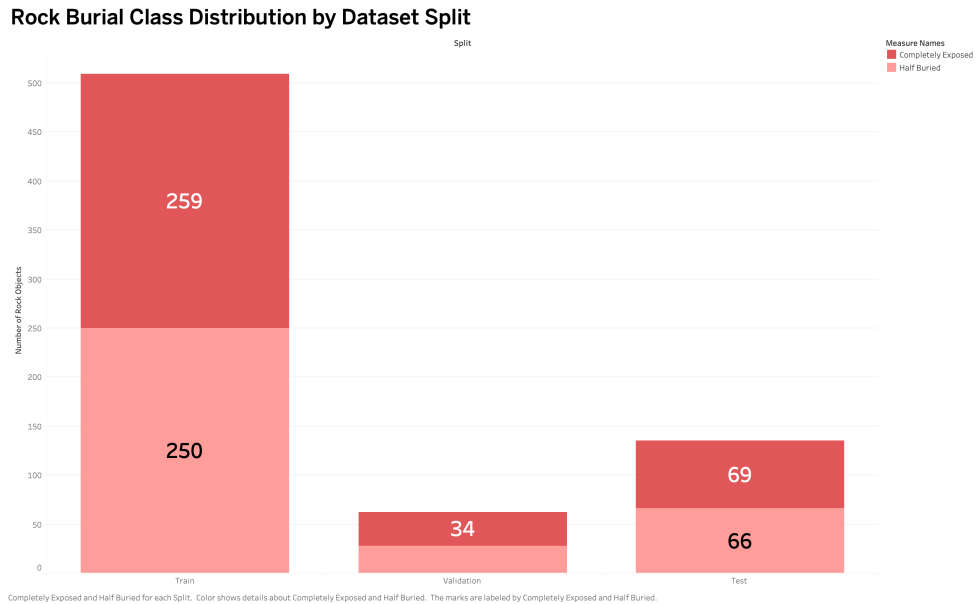


Figure 3.2: Class-level annotation counts for completely exposed and half-buried rocks in each dataset split. The closely balanced overall totals allow later performance differences to be interpreted primarily as differences in visual difficulty rather than severe label imbalance.

Training is carried out offline only: the training set has been enlarged. There are 313 training images and each image is provided with 6 photometric variants. In total, there are 1,878 augmented images, in addition to the original images. This results in 2,191 images, where 3,563 annotation instances are present in the augmented images. The augmentation approach was based on likely changes in appearance, rather than geometric changes which might pose a danger to existing boxes. For each of the 313 images, 6 photometric variants were produced, thus a total of 1,878 augmented images were obtained. There are 2,191 images in the training partition, which is shown in this figure. The enhanced data were used only for training; no changes were made to the validation and test images. The dataset-scale summary of the 448 original images, 706 annotated rock objects and two burial-status classes is provided in this chart. These data indicate a limited amount of data, which is what makes transfer learning, careful handling of held-out data and controlled augmentation so vital.

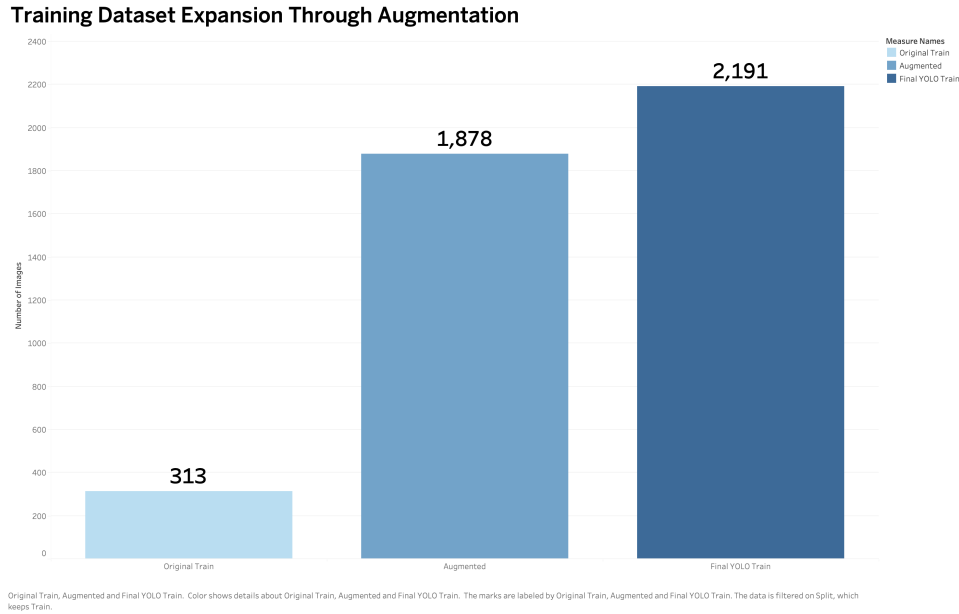


Figure 3.3: Expansion of the training partition from 313 original images to 2,191 images after six photometric variants were generated per source image. Only training data were augmented; validation and test images remained unchanged.

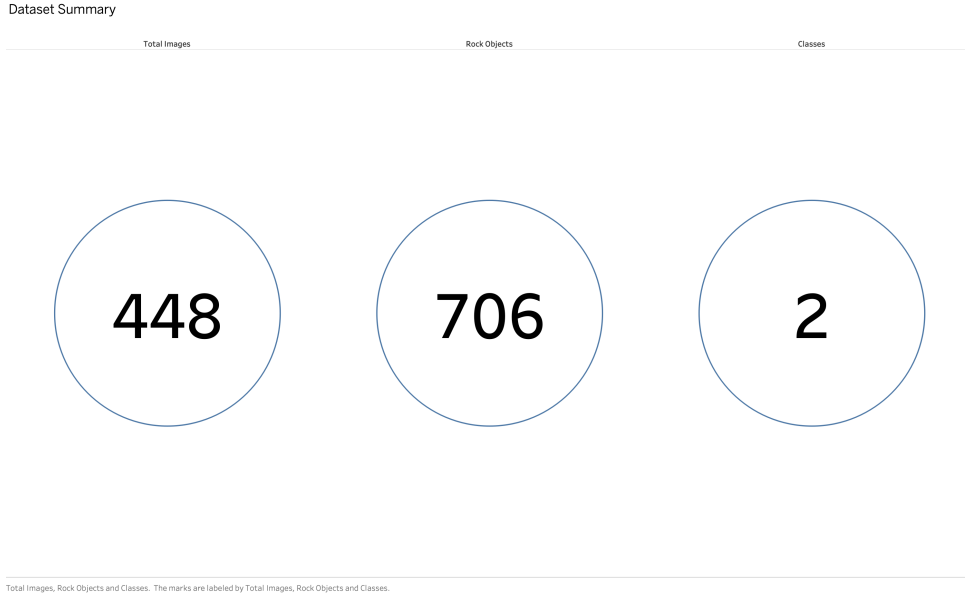


Figure 3.4: Dataset-scale summary showing 448 original images, 706 annotated rock objects and two burial-status classes. These figures define the limited-data setting that motivates transfer learning, controlled augmentation and careful held-out evaluation.

YOLOv8n, YOLO11n and YOLO11s were trained with the following main configuration: a maximum of 20 epochs, image size 896, batch size 32 and a seed of 42, automatic mixed precision, cosine learning-rate scheduling and automatic optimiser selection by Ultralytics (AdamW). The probability of horizontal flip was set to 0.5, translation to 0.04, scale to 0.1 and mosaic was turned off. Final comparative validation inference was carried out using images with 1024 pixels. The common training settings were chosen to enable the interpretability of the architecture comparison. No independent tuning of the hyperparameters that could be beneficial to one of the candidate models was performed. The training curve and the validation result of each candidate were retained and the best candidate was only selected following the common validation inference. This design isn’t as detailed as a complete hyperparameter search, though it’s a controlled comparison which is adequate for a project of this size. Table 3.2 shows the results of the validation comparison. YOLO11s had the highest F1, recall and mAP50. YOLOv8n had the best mAP50–95 score of all. As mAP50–95 was predefined to be the criterion for architecture selection, YOLOv8n was chosen. This avoids the post-hoc claim that the selected model was the “best” model according to all criteria. The precision, recall and F1 scores are shown for validation and test data

Table 3.2: Validation comparison of the detectors.

Model	Precision	Recall	F1	mAP50	mAP50–95
YOLOv8n	0.681	0.759	0.718	0.759	0.4288
YOLO11n	0.743	0.719	0.731	0.729	0.4002
YOLO11s	0.764	0.801	0.782	0.783	0.4107

for YOLOv8n, YOLO11n and YOLO11s. YOLO11s leads several operating-point metrics, which shows why it is inappropriate to use one score to decide on the detector choice. Here we show the mAP50 and mAP50–95 on both the validation and test sets of the three candidate YOLO detectors. To evaluate localisation, we used mAP50–95 with various IoU thresholds. The range was from 0.50 to 0.95, and this was a predetermined condition for the selection of the architecture.

YOLO Detection Performance — Precision, Recall & F1

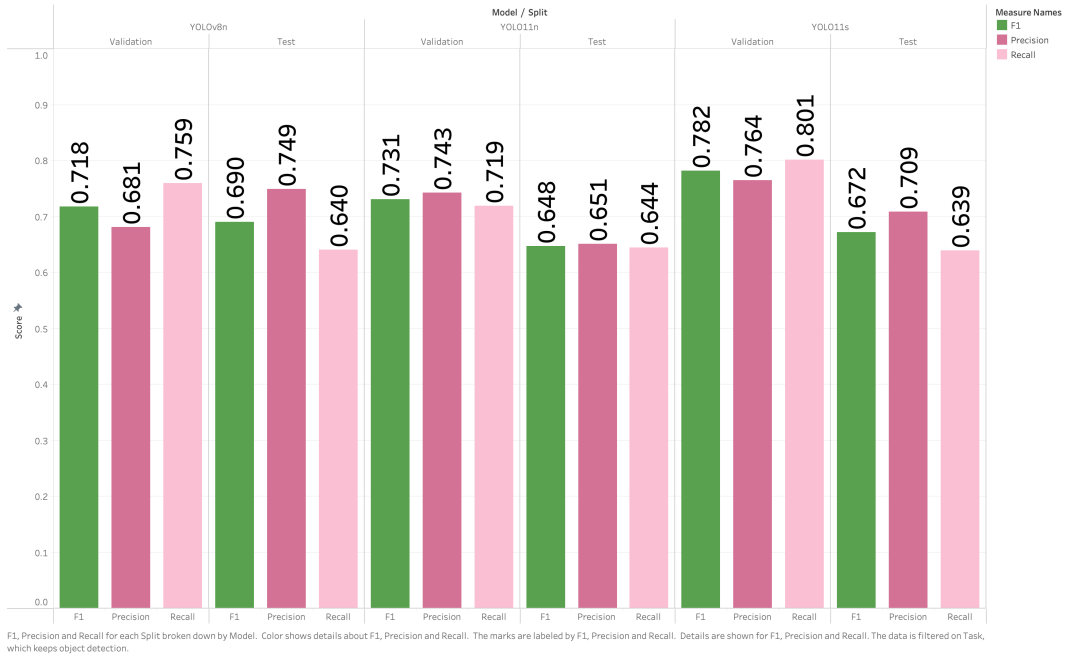


Figure 3.5: Validation and test precision, recall and F1 for YOLOv8n, YOLO11n and YOLO11s. YOLO11s leads several operating-point metrics, showing why the detector choice cannot be justified from a single score in isolation.

YOLO Detection Accuracy — mAP50 vs mAP50-95

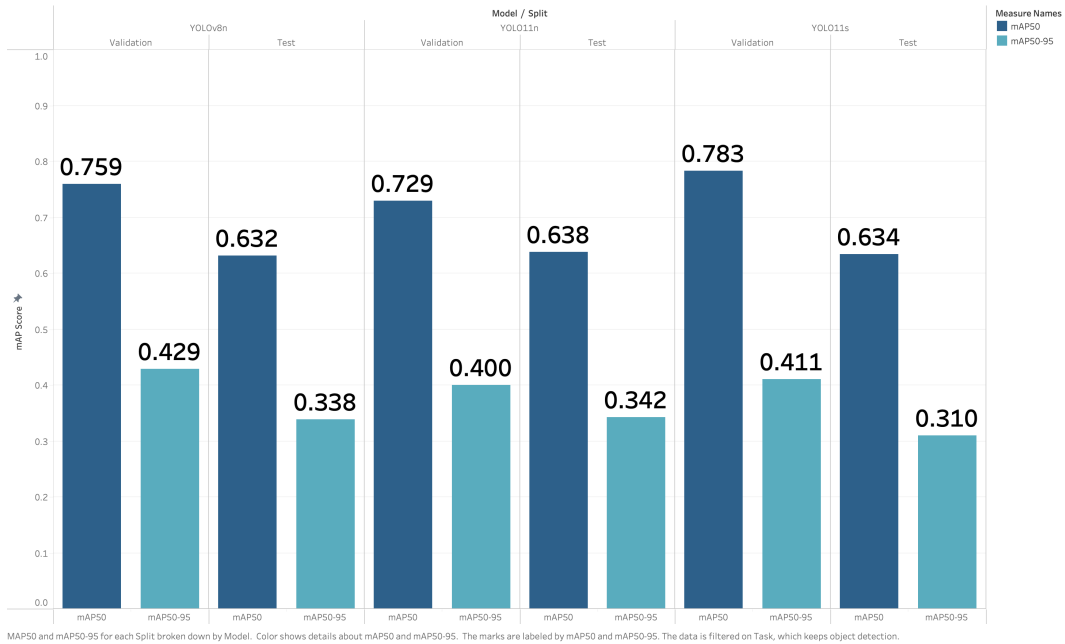


Figure 3.6: mAP50 and mAP50-95 for the three candidate YOLO detectors on validation and test data. The stricter mAP50-95 measure evaluates localisation across multiple IoU thresholds and was the predefined architecture-selection criterion.

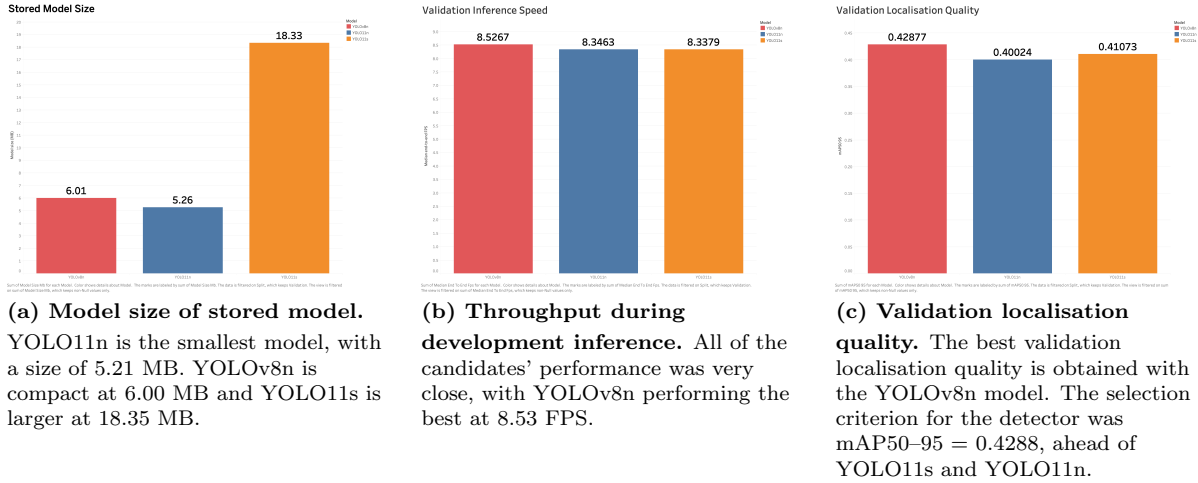


Figure 3.7: YOLO candidates validation quality and efficiency.

The YOLOv8n model selected was tested on the original architecture-selection test set, achieving mAP50–95 of about 0.3384, while validation mAP50–95 was about 0.4288. The reduction of 0.0904 (near 21% relative) indicated a significant lack of generalisation. Care has to be taken in viewing the results of all subsequent deployments. Many of the following steps were performed in a particular order. The choice of the architecture was made using validation before the 90-image test partition was investigated, so the 90 images were retained for the purpose of estimating the selected architecture at that time. Later, the same 90 pictures were used to compare edge representations and to select a runtime, so they were no longer an entirely untouched final deployment test. The dissertation distinguishes the validity of the first architecture-selection use from the later limitation introduced by runtime selection. The model-development records were also collected and stored to provide class-level output, confusion information, weights and the best score. This helped to identify the half-buried weakness before deployment. Without this evidence, lower class performance following conversion could have been taken as a quantisation effect rather than being identified as a pre-existing detector limitation. One direction of the research explored how to leverage pretrained vision–language embeddings to distinguish burial status from rock crops [14]. This work was a supporting model-development branch. A verified Raspberry Pi CLIP benchmark was not finished, and the deployments listed below only relate to YOLOv8n. The research branch used CLIP ViT-B/32 in two different ways: zero-shot classification using textual class descriptions and a simple linear probe based on frozen image embeddings. A rule for conflict resolution based on confidence was explored for the case of YOLO class vs. CLIP classification, with a CLIP override threshold of 0.80. This resulted in a field image → YOLO detection → rock crop → CLIP verification pipeline. The branch was not included in the final comparison with the Pi because running CLIP for all detected crops would add another model, more memory requirements and latency, and there was no comprehensive device benchmark to gauge the impact.

3.3 YOLO Detection and Output Processing

The end system’s main executable model is the selected YOLOv8n. For every input image, it returns a list of bounding boxes, confidence scores and class labels in one forward pass. During development, a common training and validation setup was used for the evaluation of the presented architectures. The same set of weights and the same fixed 640×640 letterboxed pixels were applied during edge evaluation in every runtime, removing confusion between detector behaviour and different image geometry. The commonly used post-processing was done by setting the confidence threshold to 0.25 and IoU (Intersection over Union) to 0.70. The candidate boxes were then transformed to a common coordinate system and assigned to either the completely exposed or half-buried class, and then stored in structured per-image outputs. These predictions drove the annotated images, class-specific counts and the precision, recall, F1 and mAP values. This helped to structure the output as evidence of detection rather than just an output image. The whole information could be linked to certain pictures and class-specific errors. The two-class detection formulation also eliminates the need to have a second classifier for the deployed benchmark: localisation and burial status are given by a single network. This reduces the complexity of the pipeline and keeps the comparison focused on one trained model. However, there is a downside to this – if the rock

is only partially visible, this can affect localisation and class assignment. In this case, detector-selection evidence and class behaviour are given greater weight than a deployment comparison based only on FPS. Keeping the YOLOv8n model weights, input-image pixels, thresholds and interpretation of the outputs the same across PyTorch, TFLite and NCNN is the main experimental control. All the changes observed can then be attributed to representation and runtime effects while the same detection task is carried out.

3.4 Edge-Deployment Design and Test Workload

The chosen YOLOv8n model was made ready in 6 model formats: PyTorch FP32, ONNX FP32, TFLite FP32, TFLite INT8, NCNN FP32 and NCNN FP16. These formats were selected as they represent the original deployment path, an exchange/runtime path, an integer-quantised deployment path and a lightweight embedded inference system. The same model weights of the chosen YOLOv8n model are used for all six representations. They were not retrained separately after export. This is one important control variable of the experiment: if there are any differences in accuracy or latency, then they are primarily due to conversion, numerical precision and backend execution, not another optimisation history. Export success was also separated from deployment success. Having a file on disk or loading it once was not sufficient to determine whether a representation was “benchmark-complete.” For comparison, the same set of 90 images was adapted to fixed-size 640×640 letterboxed images. The aspect ratio is preserved with the scale

$$s = \min\left(\frac{640}{W}, \frac{640}{H}\right), \quad (3.1)$$

with symmetric padding and a fill value of 114. It creates a common input shape without aspect-ratio distortion. There are 135 labelled rocks (69 completely exposed and 66 half buried) in the 90 images. The scale and padding offsets of the centres and dimensions of the bounding boxes were the same as for the image. The original dimensions, deployment dimensions, scaling and padding data were attached to a manifest to enable the fixed images to be traced back to their sources. The letterboxed workload was pre-generated so that different frameworks had less variation in preprocessing. All the backends were given the same pixels at the same geometry rather than relying on slightly different runtime resize implementations. These 90 images were used to create a 10-image smoke-test subset for testing model loading, input/output handling and basic post-processing before more expensive runs. It is, as such, a functional subset and not an independent evaluation data set. The loading of models, acceptance of images, inference, class-identifier mapping, generation of bounding boxes, counting per class, output of annotated images and structured JSON output were tested using the smoke-test procedure. The final functional check returned the expected single completely exposed rock on the representative image for the optimised formats. This test was carried out to verify interface compatibility, but no extra accuracy samples were taken from it. The representative data for INT8 conversion were 200 *original training images*, not validation or test images. There were 304 objects in the chosen set, balanced with 152 completely exposed and 152 half buried. This separation minimises direct leakage of the final test images into calibration and lets both classes influence the estimation of the activation range. There were 304 labelled objects in the calibration set. Equal numbers of objects by class do not mean that the calibration data are statistically the same as the test distribution, but the design ensures that no single burial class is more represented due to a higher number of representative objects. Following conversion, the calibration images were not needed for device inference. Provenance was retained using a calibration manifest rather than using the calibration subset as a second evaluation set.

3.5 Raspberry Pi Environment and Tooling

The target device was a 64-bit ARM Raspberry Pi 5 Model B with around 8 GB RAM [16]. The OS was Debian GNU/Linux 13 and Python 3.13.5. The major versions of the packages installed were Ultralytics 8.4.115, ONNX Runtime 1.28.0 and OpenCV 5.0.0, NumPy 2.5.1, Pandas 3.0.5 and psutil 7.2.2. LiteRT inference used the `ai.edge.litert.interpreter` interface. The FP32 and FP16 NCNN variants were performed using NCNN. The official Raspberry Pi 27 W USB-C power supply (rated at 5.1 V/5 A) was used for power. For the final setup there was no Active Cooler, and the issue of cooling is discussed in the Raspberry Pi documentation for sustained high-load operation [15]. There was no live camera used. Only a MacBook was used to control and transfer files through SSH/SCP; the measured inference was done with the Pi CPU. These exact versions are crucial since the behaviour of inference does not rely on the model file alone. It may be affected by runtime releases, Python bindings, numerical libraries or operating-system

Table 3.3: Important Raspberry Pi benchmark environment.

Component	Recorded configuration
Device	Raspberry Pi 5, aarch64, approximately 8 GB RAM
Operating system	Debian GNU/Linux 13
Python / Ultralytics	Python 3.13.5 / Ultralytics 8.4.115
ONNX Runtime	1.28.0
OpenCV / NumPy / Pandas	5.0.0 / 2.5.1 / 3.0.5
Resource monitoring	psutil 7.2.2
LiteRT interface	<code>ai_edge_litert.interpreter</code>
Power	Official Raspberry Pi 27 W, 5.1 V/5 A
Cooling	No Active Cooler
Input source	Stored images; no camera

changes, which may impact performance or compatibility. The environment table is therefore considered an integral part of the reproducibility evidence, not only software documentation. The deployment scripts loaded the models, read the images, made the inferences, performed post-processing, sampled resources and produced output in CSV/JSON format. The output manifests and saved predictions were used to ensure that each completed run contained the intended 90 images rather than silently failing. The tooling was broken down by job. Environment verification involved recording software versions. A single-image inference provided a low-cost functional test; the benchmark produced per-image timing/resource records and aggregate outputs; and a dedicated stability script was used for repeated continuous inferences. Files were transferred between the MacBook and Pi using SSH and SCP without moving the inference workload off the device. This separation reduced the manual effort and made it easier to distinguish between an environment problem, a model/interface problem and a sustained-runtime problem. The load time of the models was measured and model loading was not included in the per-image inference measurements where possible. Warm-up calls were given before the timed pass. CPU and process RSS were used as system-level indicators rather than energy measurements. The Raspberry Pi monitoring interface was used to record the temperature. These measurements thus describe the actual software infrastructure running on the Pi and not an abstract hardware-only cost of the neural network.

3.6 Benchmark and Evaluation Protocol

The final benchmark used 640×640 inputs, a batch size of 1 and a confidence threshold of 0.25, with IoU set to 0.70. Each representation was given three warm-up inferences, followed by one timed pass over the 90 images. This did not follow the previous plan shown in Table 3.4. The planning document had stated that there would be 10 warm-ups, 3 timed passes and a 30-minute stability test. The planned settings were not carried out and are not represented as evidence. In each successful representation, the

Table 3.4: Planned protocol versus actual protocol.

Element	Earlier plan	Executed benchmark
Input size	640×640	640×640
Batch size	1	1
Warm-up	10 inferences	3 inferences
Timed evaluation	3 passes	1 pass
Images per completed format	90 per pass	90
Stability target	30 minutes	Separate short tests only
Completed formats	Planned six	Five complete; ONNX aggregate N/A

measurements obtained constituted benchmarks for inference time, throughput, CPU utilisation, process resident memory, temperature and predictions. The detection results were then compared to the common labels to get precision, recall, F1, mAP50, mAP50–95 and count MAE. The final comparison is based on the median FPS as this is less influenced by extreme timings. The latency for each image was measured after warm-up, and the average and distribution of individual timings were calculated. The observed

inference rate was used for determining throughput instead of the theoretical number of operations. Process memory was considered as RSS (resident set size), the memory that the running process is using, but not all system allocations indirectly connected with the workload. The other software/runtime measurement is CPU percentage. A backend can reduce latency with increased utilisation of available cores. Thus, high CPU utilisation is read in conjunction with latency and is not penalised automatically. When saving class labels, confidences and bounding boxes were saved with them for downstream accuracy validation. Rock count was determined by the number of detections that were retained in an image, making count MAE traceable to the same predictions as mAP. Temperature and throttle indicators were collected during the benchmark/stability runs so that a high-FPS result could be checked for signs of unsustainable operating conditions. To test accuracy, the predictions saved in each of the completed representations were used, instead of intermingling timings with detections from a different workload. This enabled the same labels to be used for both detection metrics and per-image rock-count error. It also allowed failure to be observed: a representation that failed to produce all predictions could not be given an aggregate row estimated from only a few data points.

3.7 Reliability, Diagnostics and Reproducibility

Five representations hit the 90-image mark: PyTorch FP32, TFLite FP32, TFLite INT8, NCNN FP32 and NCNN FP16. ONNX FP32 was properly exported and was able to run on isolated images. Repeated sustained attempts resulted in Raspberry Pi reboots before a valid aggregate run was created. There is no way to determine the root cause, and hence ONNX aggregate latency, accuracy or resource values are not estimated. The only defensible conclusion is that ONNX FP32 was able to make inferences in isolation but did not complete sustained benchmarking in the tested configuration. Any indication of an application exception and/or system reset was checked in the diagnostic log. The historical data revealed a temporal relationship between continuous ONNX attempts and the sudden reboots, but were not able to identify thermal throttling, undervoltage, out-of-memory termination or an ONNX Runtime problem. This is the reason why ONNX is reported as N/A in the final benchmark table instead of reporting results from only part of a run. Reporting its evidential limitation is better than presenting an incomplete row. The other difference was between completing a fixed amount of work and remaining stable continuously. NCNN FP32 completed the 90-image benchmark, but a later continuous run hard-reset the Pi and did not generate a valid continuous CSV file. TFLite INT8 completed a real continuous experiment lasting around 2 minutes and generated about 2,533 inference rows at approximately 22.7 FPS. The temperature rose from the mid-40s to 76–77 °C. The maximum temperature recorded was about 77.1 °C, with no throttling or reboot reported. This is definitely short-term evidence. No valid 10-minute or 30-minute final stability runs were conducted. A file containing a reference to a test that was 30 minutes long was found; however, it turned out to be a duplicate of the shorter time series and was not regarded as an independent data source. This separation of planned, completed and invalid evidence is part of the reproducibility approach and should not be hidden. To ensure repeatability, the deployment folder structure, environment records, input manifests, annotated outputs, benchmark CSVs and prediction JSON files were saved. The evidence was also checked to ensure experiments were not duplicated, so that more than one result folder wasn't included as multiple experiments. If a filename conflicted with its contents, the contents and the number of rows were given priority over the filename. This is the evidence rule that led to the dissertation reporting one actual approximately 2-minute stability run and no completed 10- or 30-minute final run. The same treatment was applied to backup folders. A copy of a benchmark was considered to be a backup, not another trial run. The number of annotated images and prediction JSON files was checked to ensure the correct number of frames were produced, e.g. 90 frames with NCNN FP16. By this means the number of files in an archive will not be misinterpreted as representing the number of independent trials.

The project repository containing the implementation and reproducibility materials is available at <https://github.com/tanishkshindepatil01/Edge-Deployable-Rock-Detection-and-Burial-Status-Classification>.

3.8 Chapter Summary

The implementation resulted in a controlled path from an empirically chosen YOLOv8n detector to five fully benchmarked Raspberry Pi representations and an incomplete ONNX format under sustained testing. The next chapter discusses what those measurements represent, especially the half-buried weakness

and the balance between PyTorch accuracy, INT8 speed and NCNN FP16 accuracy retention.

Chapter 4

Critical Evaluation

4.1 Evaluation Overview

The outcome of central evaluation is that there is no single performance indicator for deployment quality. The slowest representation had the strongest measured detection quality, and the fastest representation had a measurable accuracy penalty. Access to resources and use of resources (utilisation and operational) There were also some differences in behaviour between the backends. The key benchmark evidence is summarised All benchmarks were consolidated in Table 4.1 as a single table.

Table 4.1: Raspberry Pi benchmark results – in canonical format. FPS is the median throughput.

Representation	Latency ms	FPS	CPU %	Peak RAM MB	P	R	F1	mAP50	mAP50-95
PyTorch FP32	397.48	2.52	44.47	839.47	0.716	0.550	0.622	0.548	0.275
TFLite FP32	143.70	6.95	69.95	868.27	0.669	0.542	0.599	0.535	0.270
TFLite INT8	44.13	22.62	62.83	824.22	0.661	0.539	0.594	0.503	0.234
NCNN FP32	75.09	13.26	83.32	835.25	0.667	0.542	0.598	0.535	0.271
NCNN FP16	72.31	13.82	87.08	835.45	0.672	0.542	0.600	0.537	0.270
ONNX FP32	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A

4.1. EVALUATION OVERVIEW



Figure 4.1: Three-image graphical summary of Table 4.1.

Count MAE was 0.600 for PyTorch FP32, TFLite FP32 and NCNN FP32, 0.611 for NCNN FP16 and 0.633 for TFLite INT8. The close values suggest that there was no catastrophic counting effect due to quantisation, The slightly higher INT8 error is consistent with its lower detection metrics.

4.2 Detector Selection and Class-Specific Behaviour

The explanation of the results of detector selection should be given against the criterion. YOLO11s had the highest validation mAP50 (0.783), recall (0.801) and F1 (0.782), whereas YOLOv8n had the highest validation mAP50–95 (0.4288). It was therefore internally valid to choose YOLOv8n, since the main criterion was set as mAP50–95. It would be wrong to say that YOLOv8n led on all measures. Another reason for not equating validation leadership with finality is generalisation. The validation result of the mAP50–95 of YOLOv8n was reduced from 0.4288. To approx 0.3384 on unseen test set. The absolute change was 0.0904 (or approximately 21% relative change). This implies that there is material uncertainty beyond the development partition. This corresponds with literature centered on generalisation under varying farming conditions as a problem to be continued. [1, 2]. This could be because of two factors: the size of the validation partition could be too small to provide a good estimate, or because the validation partition may not be representative of the entire dataset. The images that were not used may be more complex, or the chosen training recipe might not be suitable. cover all variations of soil, illumination and occlusion. The experiment is not designed to isolate which explanation dominates. Therefore, the result of validation is the correct answer. was a bit more cheerful than the (held-out) test, although, naturally, there was no specific data or optimisation problem. The experiment did not show which factor caused the reduction. The class-specific semantic deficiency was the most serious. At the evaluated source-detector operating 64/69 of all fully-exposed rocks were correctly represented (about 92.8%), compared with only 23 of 66 half-buried rocks (about 34.8%). These are The confusion analysis is used to obtain rates, rather than AP values. They're significant because the weakness of the burial was present prior to conversion to Raspberry Pi. INT8 and FP16 can change the confidence or localisation, but cannot be taken as the only reason for a weakness already evident. in the source detector. This is immediately relevant to the design: if future effort focused only on runtime optimisation then this result would be directly applicable to the design. Increase the FPS without changing the main application disadvantage. Better: half-buried examples, better hard negative coverage, and maybe sensing. modalities less sensitive to soil/stone appearance are likely to provide more field value than Further micro optimisation of a fast INT8 runtime. This is also a determining aspect in the aggregate mAP. An average over two classes can make it obscure that one class is close to the desired behaviour, and the other is. frequently missed. A rock that is not found or is only partly dug up can be more expensive than A minor exposition error of a fully exposed rock. Future evaluations should, therefore, Use per-class AP/recall and field relevant miss rates and consider The question of whether class-specific confidence thresholds are justified.

4.3 Accuracy–Efficiency Trade-offs Across Edge Formats

TFLite FP32 decreased the mean latency by about $2.77\times$, NCNN FP32 by $5.29\times$, NCNN FP16 by $5.50\times$, and TFLite INT8 by $9.01\times$, compared to the PyTorch baseline of 397.48 ms. $2.77\times$, NCNN FP32 by $5.29\times$, NCNN FP16 by $5.50\times$, and TFLite INT8 by $9.01\times$. All deployment formats that finished the Raspberry Pi benchmark are plotted on the graph below. The graph below shows median throughput vs mAP50–95 for all deployment formats that made it to the end of the Raspberry Pi benchmark. Faster points are on the right and higher points better localised. The following model is not included because it failed to successfully complete an aggregate benchmark: onnx_fp32. For each deployment format, the median throughput and mean inference latency across deployments is shown. TFLite INT8 is the fastest version with 22.62 FPS and the lowest mean latency (44.13 ms). F1, mAP50 and mAP50–95 are computed for the 5 deployment formats that completed the 90-image benchmark, and compared to each other.

Overall, PyTorch shows the best accuracy performance whereas INT8 shows better inference speed with lower localisation accuracy. PyTorch FP32 had the best edge-benchmark mAP50 (0.548) and mAP50–95 (0.275), However, the median of 2.52 fps is limiting for this on-the-go vision part. The majority of that accuracy (mAP50 0.535) along with an increase in throughput to 6.95 FPS. NCNN FP32 yielded nearly the same results. same accuracy (mAP50 0.535; mAP50–95 0.271) at 13.26 FPS, demonstrating that runtime choice can provide a significant boost in speed without a substantial loss of measured accuracy. NCNN FP16 produced mAP50 0.537 and mAP50–95 0.270 at 13.82 FPS. Its The

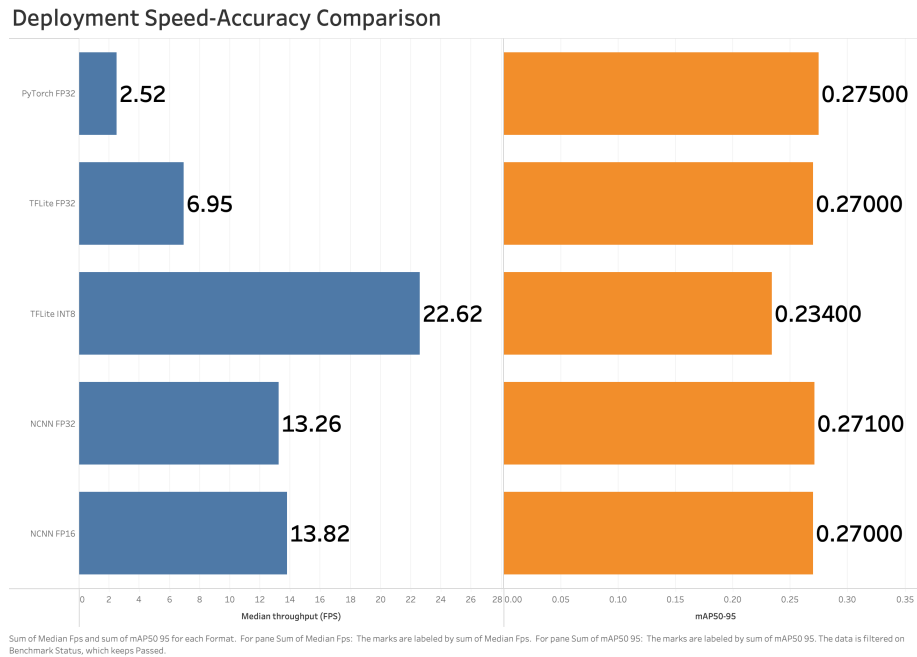


Figure 4.2: Median throughput plotted against mAP50–95 for each deployment format that completed the Raspberry Pi benchmark. Points farther right are faster and points higher on the chart retain stronger localisation quality; ONNX FP32 is omitted because no valid aggregate benchmark was completed.

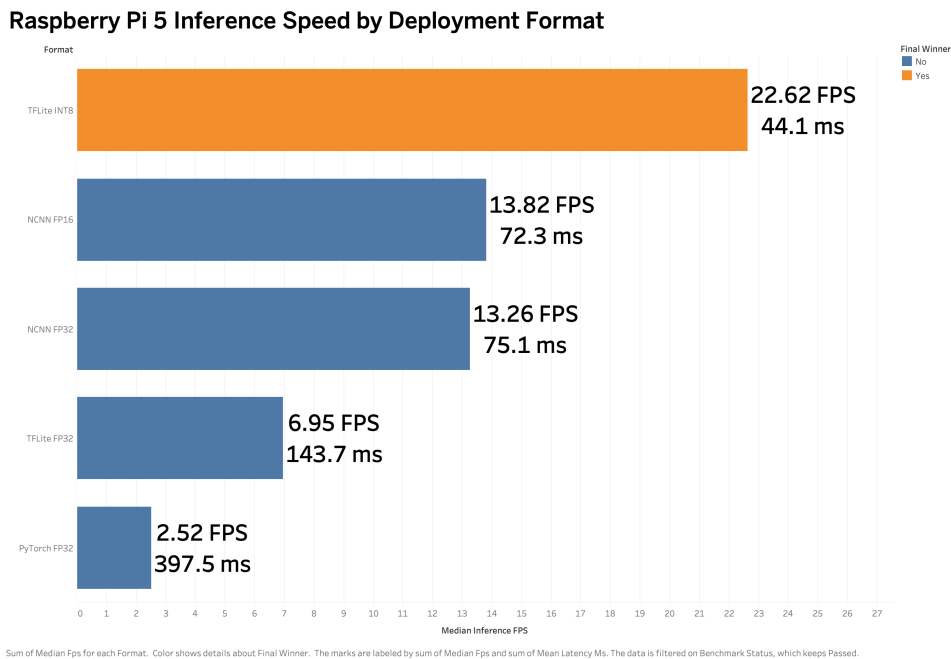


Figure 4.3: Median throughput and corresponding mean inference latency for each completed deployment format on Raspberry Pi 5. TFLite INT8 provides the highest speed at 22.62 FPS and the lowest latency at 44.13 ms.

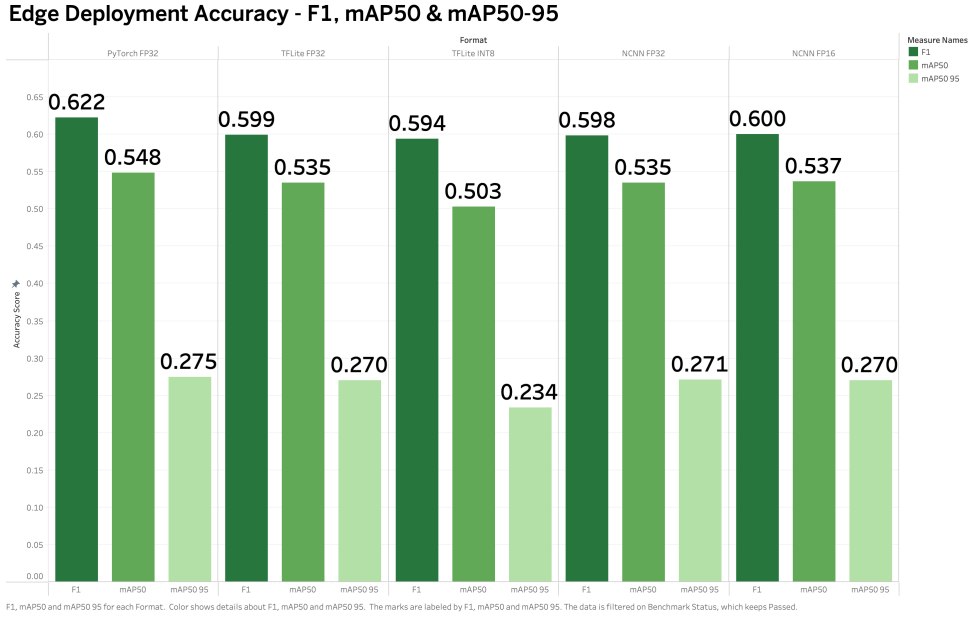


Figure 4.4: F1, mAP50 and mAP50–95 for the five deployment formats that completed the 90-image benchmark. PyTorch retains the strongest aggregate accuracy, while INT8 trades measurable localisation quality for substantially faster inference.

model storage decreased by around 49% when compared to NCNN FP32, and the latency decreased by around 3.8%. This is important because it shows an example which is opposite to the notion that half the number needed to stretch the same width. CPU speedup. Using this Raspberry Pi/runtime setup, we made huge savings in storage when compared with the execution-time reduction. Notably, all three deployments without quantisation are pretty close to each other. TFLite FP32 and NCNN FP32 had an mAP50 score of 0.535, while NCNN FP16 had 0.537; all three were within 0.013 mAP50 of the PyTorch baseline. Their mAP50–95 values (0.270, 0.271 and 0.270) were also close to PyTorch at 0.275. If this experiment is conducted with the resolution of these, the results are within the same range as the results of this experiment. representations retained most of the detector quality. This further supports the argument The idea that some bigger than expected speed improvements can be achieved before taking the bigger. quantisation penalty of INT8. The precision and recall of the three formats was slightly different. PyTorch precision was 0.716 and recall 0.550; the converted FP32/FP16 formats clustered around 0.667–0.672 precision and 0.542 recall. The precision achieved by TFLite INT8 was 0.661 and the recall was 0.539. The direction is uniform. The lowered detector behaviour is not a complete change in behaviour, but rather some degradation in detector behaviour. The lower mAP50–95 indicates that stricter localisation quality is more sensitive than the headline operating-point recall suggests. The biggest performance change was for TFLite INT8 with a mean latency of 44.13 ms and median throughput of 22.62 FPS, around 9 times as fast as PyTorch. The loss was in terms of mAP50 going from 0.548 to 0.503 (0.045 absolute, about 8.2%) and in mAP50–95 from 0.275 to 0.234 (0.041 absolute, about 14.9%). This is a typical quantisation trade-off: Inference can be cheaper if integer arithmetic is used, Approximated weights and activations can help reduce task accuracy [6]. So, the big question remains, does this loss make sense for the desired Stage of the system. The drop observed in mAP is genuine and should not be taken for granted, although INT8 is still useful for applications requiring low latency. is a required first class. The FP32 runtime comparison is useful because it was done with the numerical precision of the two different types of floating point units stripped out. from backend implementation. All three models use nominally full PyTorch FP32, TFLite FP32 and NCNN FP32. precision, yet mean latency changes from 397.48 to 143.70 to 75.09 ms. Much of the There are two phases of speed-up: firstly, the inference engine, and secondly, the graph/runtime optimisation before reduced precision is introduced. INT8 has a bonus to this run-time effect. This staged interpretation is more defensible than attributing the whole 9.01× improvement to quantisation alone. It also shows a need to compare throughput in a batch-one scenario. Large batches are not representative of a stream from a camera, but they can be used to utilize hardware resources on a server. In which each frame requires a prompt answer. The INT8 representation nominally has a median FPS of 22.62, The 2.52 FPS is quite slower than the latter, which

processes at much faster speed. PyTorch baseline. This will be sufficient depending on the field of view of the camera, The overlap between observations and vehicle speed were not measured here. The study is therefore not designed to ensure the maximum tractor speed. The same caution should be taken with accuracy differences as with the estimates based on 90 images rather than an exact. population values. The decimal measurement in the report will help when comparing completed runs on a common set, although the experiment was not large enough to prove that there was a difference of a few The research results of thousandths between TFLite FP32 and NCNN FP16 would be replicated on separate farms. The broad Patterns are more defensible: PyTorch is the slowest, and the most accurate overall; INT8 is much more efficient with a clear accuracy penalty; and NCNN formats preserve the majority of the FP32 accuracy while providing intermediate throughput.

4.4 Resource, Counting and Thermal Behaviour

Peak process memory was not as much different than latency. The lowest measured value was for TFLite INT8, at 824.22 MB. The range for PyTorch, NCNN FP32 and NCNN FP16 is 835–839 MB. TFLite FP32 reached 868.27 MB. In contrast, there is no major memory savings for each of the compact numeric types. Helps consolidate idea that the libraries, buffers and interpreter overhead make up the runtime libraries of RSS Other than model parameters, in addition. Counting behaviour is an application oriented check. The three FP32 implementations had a count MAE of 0.600, NCNN FP16 0.611 and TFLite INT8 0.633. The small spread is not an indication that individual errors can be substituted. A rock that was not completely dug up. If another false-positive exposed rock can partially cancel in image level count, but represents two detection mistakes. In this case, qualitative assessment of dense or ambiguous scenes can be useful when diagnosing, why the number of errors varies. It goes for CPU utilisation too, there is a profile, and it varies per run. PyTorch FP32 averaged 44.47%, TFLite FP32 69.95%, TFLite INT8 62.83%, NCNN FP32 83.32% and NCNN FP16 87.08%. A higher CPU percentage does not necessarily indicate poor performance if it is accompanied by much lower latency; it represents the degree of aggressiveness in the backend’s use of processing power. The NCNN formats had the highest average CPU utilisation, while TFLite INT8 achieved the highest throughput without the highest CPU utilisation. This is especially important for integration of systems. Less capacity might be available with a CPU-intensive runtime for camera handling, GPS processing, communication and control logic, even if raw inference latency is desirable. But PyTorch unused CPU does not make it, If that through-put is not high enough, preferable. Therefore, an end-to-end solution is required in the future, namely: after all concurrent components are installed evaluate overall application use, Instead of making an assumption that the isolated detector profile remains constant. No thermal throttling was observed with the 90-image benchmark. Approximate maximum temperatures were 55.10 °C for TFLite INT8, 56.75 °C for NCNN FP32, 59.50 °C for NCNN FP16, 61.70 °C for PyTorch and 66.10 °C for TFLite FP32. These maximum temperatures will be short run and will not be equilibrium temperatures. The temperature increased from the mid-40s to approximately 76–77 °C during the continuous test. The system achieved 22.7 FPS over about 2,533 recorded inferences. No throttle down or reboot was detected but, the upward thermal trajectory implies that the result only demonstrates short-duration stability. The performance of Raspberry Pi 5 has been significantly enhanced over the previous versions. as documented, cooling is critical for long operations. [16, 15]. A controlled active-cooling experiment is therefore a helpful next step, rather than stating that the absence of a cooler caused any particular failure observed here. Why the 90 image temperatures are different from the continuous INT8 trace is also reflected in this difference. Short benchmarks cannot be used for determining thermal sustainability. Once the fixed workload is completed, INT8 was almost done. 55 °C, but repeated inference continued and the temperature came to the high 70s in about. two minutes. A test format that conducts a brief test at a moderate temperature could behave very differently after a few minutes. The CPU frequency and the throttling. In addition to temperature, flags should be logged when testing is conducted over an extended period of time to distinguish degradation in performance from A straightforward increase in the temperature of the sensor.

4.5 Operational Stability and Runtime Failures

The negative runtime results are analytically significant because a deployment format which can not be run at runtime is not useful. The ability to make isolated inferences does not adequately measure the ability to complete desired tasks. ONNX FP32 Successfully achieved isolated inference, indicating that it

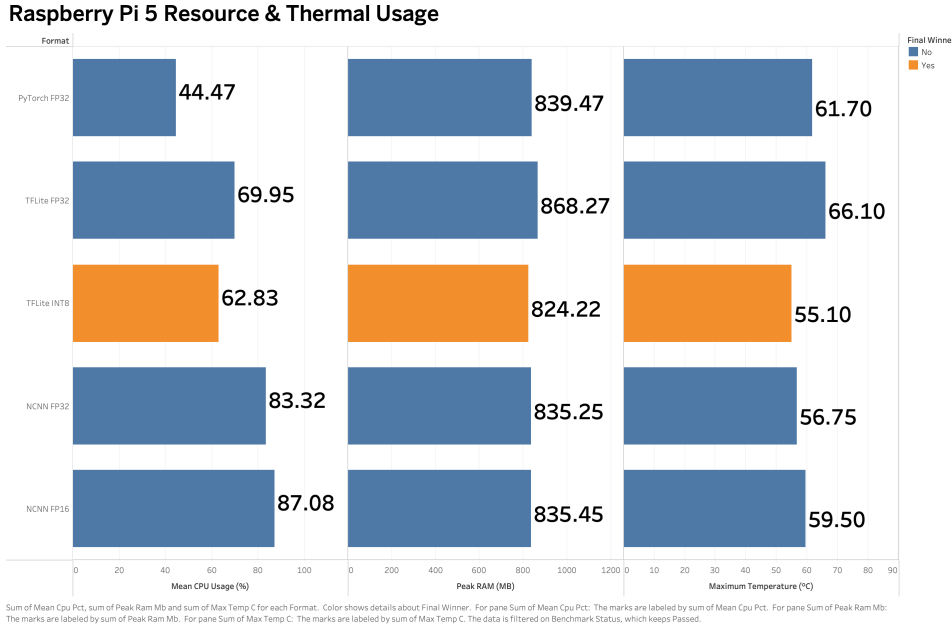


Figure 4.5: Mean CPU usage, peak process memory and maximum short-run temperature for each completed format. Faster inference does not translate uniformly into lower CPU load, memory demand or thermal output.

is possible to export and execute basic inference. But repeated requests caused inexplicable restarts. Since there is no valid 90-image ONNX benchmark available, There are a number of possible reasons, including thermal, power, memory or runtime behaviour, but no details of this in the logs. a particular cause. One would be beyond the evidence. NCNN FP32 is different in terms of the problem. It passed the baseline test of 90 images and is therefore considered to be passing. Has aggregate metrics, but a subsequent continuous test resets the Pi. Short benchmark This is not as good as repeated sustained operation for completion. TFLite INT8 is the only format for which a full 90-image benchmark and the genuine close to two-minute continuous experiment are available. These evidences can be clearly articulated. File conversion is successful when the file is exported, a single-image smoke test is successful and sets up the basic functional compatibility; a full 90-image benchmark provides short-workload aggregate behaviour; and a continuous test gives evidence of a sustained operation. Do not consider these levels as synonymous. ONNX reached the single-image level, NCNN FP32 reached the fixed-benchmark level, but failed a later continuous test, and INT8 reached the available continuous-test level. NCNN FP16 has strong fixed-benchmark metrics but was not continuously validated to the same level. These outcomes align with the overall principle of benchmarking that representative workloads and outcomes provide a good basis. Important measurements are system-level - don't take it for granted that portable model formats behave the same on a particular device [17]. They also can assist with non-reporting. provides results based on observed values of the aggregates, not based on assumed missing values of the aggregates.

4.6 Deployment Selection

TFLite INT8 provides the strongest combined evidence for the tested Raspberry Pi 5 with this CPU-only setup. It has the lowest mean latency of 44.13 ms, highest median throughput of 22.62 FPS and lowest The best achieved of 824.22 MB of peak process memory for the completed formats, together with The 90 image run was successful and it had the only completed continuous-inference test. So the selection is made on the basis of Available stability evidence and resource profile—in addition to speed. Recommendation is based on context. The accuracy of TFLite INT8 is lower than the FP32 and Alternatives to FP16, and a different hardware accelerator, cooling configuration, workload or The preferred point might change if the accuracy requirement changes. NCNN FP16 is the top secondary candidate for accuracy retention is more important: it went up to 13.82 FPS and mAP50 0.537, The PyTorch mAP50 score is 0.548, which is very close to the original PyTorch score. In this study, one of its drawbacks is that there is no equivalent completed continuous stability test. A Pareto type decision can be taken as a final choice.

PyTorch occupies the high-accuracy/low-speed end, INT8 is at the high-speed/lower-accuracy end, and NCNN FP16 lies between them. None of the representations dominates on all measures. The chosen point Reflects edge computing and evidence that was completed on the device. The decision boundary may be at a different level if the loss of a rock in the downstream system were very high. shift towards NCNN FP16. This decision could be tested with other priorities. If mAP50 retention was taken into account If the ultimate aim was to minimize the absolute loss, then NCNN FP16 would be attractive, as it only loses 0.011 in absolute mAP50 from PyTorch, while attaining more than $5\times$ the throughput. When maximum throughput and available continuous evidence dominate, INT8 is better. If simplicity and fidelity to the PyTorch doesn't have to convert, but it does not meet the speed requirement. This indicates that this recommendation is not a "post hoc" weighted average made after the fact. After seeing those results, having an explicit prioritisation of edge responsiveness was certainly a good thing. accuracy constraint. Such priorities would have to be embedded in a decision by the agricultural operator about production. In some instances, the effects of undetected rocks may be directly correlated to be damage to the machinery, while a false positive may result in an unnecessary stop or removal attempt. This cost is not available. In the present project, the dissertation does not convert mAP and count MAE into an unsupported economic, it's just simple and uncomplicated. The following recommendation is technical/Configuration specific. The same word of warning applies to the 22.62 FPS number. Inference throughput is just one Part of a future field pipeline. A production system would also need to decode camera frames, It would also require timestamp handling, coordinate transformation or association, detection logging and possibly communication with a control system. The CPU and memory requirements for those operations. This does not include a benchmark for stored-image. The measured value should thus be taken as available The detection throughput of the test workload, rather than a guaranteed end-to-end frame rate after all future components are integrated.

4.7 Threats to Validity and Relation to Existing Work

There were several limitations that restricted the conclusions that could be drawn. Firstly, the evaluation was carried out using a single Raspberry Pi 5 rather than repeated devices, so manufacturing variation and device-to-device thermal behaviour are not represented. Secondly, the benchmark that was executed had 3 warm ups and Rather than multiple independent timing runs, one timed pass. These variance and tail latency can therefore be However, not as a significant distribution over repeated experiments, but rather as a report from that pass. Thirdly, the test set of 90 images was not involved in the selection of *architecture* (but was used later). To compare and select the end edge runtime. There is no other "clean" data set that would offer the same. The final estimate without runtime selection, which is unbiased. Fourthly, a workload was stored in the letterboxed. images as opposed to a live-camera pipeline. Capturing images, decoding them, buffering them and field If the motion does occur, it may impact end-to-end latency. Fifthly, the evidence of stability is not balanced: INT8 runs for a total of about two minutes and NCNN FP32 has a failed continuous attempt. and the other formats do not have similar long repeated tests. Sixthly, accuracy is influenced by the choice of the confidence value and IoU. Alternative operating Some points may vary in precision/recall and error count – particularly for rocks that are partially buried. Finally, GPS mapping, size/weight estimation and robotic removal are not validated by the project. It should be treated as a focused deployment experiment, not an autonomous system that is ready for use in the field. This positioning is consistent with feedback about the value of Raspberry. The use of Pi as a low-cost agricultural platform is one, while another is the need for wider-domain and real-world validation [7], together with unresolved accuracy/resource trade-offs [20]. Plus, there are differences between internal and external validity. Internal comparability is strengthened by fixing the images, detector and thresholds over all completed runtimes. External The results are from one board, one data set domain, and one stored-image, making their validity lower. pipeline. The study is best for answering the question, "which representation worked best in this controlled configuration?" and is less strong in answering the question "which representation will be best on all agricultural edge systems?" The latter claim is not made. The other interpretive boundary is the selection of the test set. The 90 pictures were correctly untouched during detector architecture selection, but the same images were later used to compare runtime representations and choose TFLite INT8. So, the reported INT8 metrics refer to the selection workload and not a new post-selection estimate. In a more detailed experiment, the entire detector-plus-runtime would be frozen. Evaluate the pipeline and assess using an independent and fresh data set. This will determine if this is actually the case. The measured compromise and half-buried weakness continue after the data has been used for the final edge. choice. The outcomes do, however, concur with a few themes in related work. Long has been the time integer quantisation has been around.

Most recently motivated as a way to do efficient inference on resource-limited processors. [6]. There is a significant speed-up on the actual target Pi as the actual INT8 result highlights, At the same time, the non-zero accuracy cost is displayed. In the case of MLPerf: the YOLOv8n detector will run at a range of around 2.5 FPS to 22.6 FPS, depending on representation. [17]. The fact that the half-buried weakness is still present in the agricultural context is consistent with wider reviews that highlight variability in scenes and dataset coverage as significant problems. [1, 2]. The project also facilitates research with alternative sensing. Thürkow et al. investigated the The detection of stones in agricultural land by thermal imaging [22], and Proposing an approximate path when there is a low degree of RGB contrast. The present study does not establish whether thermal or depth sensing would resolve the burial-status issue, but it provides a real justification to test such modalities. The main obstacle that remains is semantic visibility of partially buried stones. not just a lack of CPU power.

4.8 Overall Answer to the Research Question

Two-class YOLOv8n detector can be effectively used to process the stored-image on the Raspberry Pi 5. CPU-only inference, but results are highly dependent on representation. PyTorch FP32 has the greatest overall detection quality of completed edge runs and is slow at 397.48 ms mean latency and 2.52 median FPS. TFLite INT8 reduces the mean latency to 44.13 ms and raises throughput to 22.62 FPS, while mAP50 decreases from 0.548 to 0.503 and mAP50-95 from 0.275 to 0.234. NCNN FP16 is in the middle at 13.82 FPS and mAP50 0.537. The largest source of accuracy error is not solely a conversion artefact: half-buried rocks were much more difficult to identify than completely exposed rocks in the source detector. When it comes to the results, five representations passed the fixed benchmark, and ONNX failed to produce a valid aggregate run. The available short continuous experiment was also finished with TFLite INT8 without recorded throttling or reboot, although the temperature was reported to be around 76–77 °C. The research answer to the question is not a universal “winner”, but rather a trade-off: TFLite INT8 is the preferred format for the tested proof-of-concept configuration, while NCNN FP16 remains an accuracy-retention alternative that requires stronger stability validation.

Chapter 5

Conclusion and Future Work

5.1 Overall Conclusion and Answer to the Research Question

The 2-class model of the detector of the rock burial status (YOLOv8n) was proven to be effective. The model can be moved from the development phase to inference phase in the CPU of Raspberry Pi 5. The project has been broken down into a common 90 image edge workload, created 6 runtime representations and is completed with full aggregate results for 5 of them. Even if the trained detector is frozen, representation is playing a significant role in practical deployment as evidenced by the evidence. The highest edge aggregate accuracy is provided by PyTorch FP32 with mAP50 0.548 and mAP50-95 0.275 but with 397.48 ms mean latency and 2.52 median FPS. The mean latency of TFLite INT8 is 44.13 ms with a throughput of 22.62 FPS, which achieves $9.01\times$ speedup compared with PyTorch FP32, at the cost of mAP50 reducing from 0.548 to 0.503 and mAP50-95 reducing from 0.275 to 0.234. NCNN FP16 has been proven to be quite satisfactory in terms of good compromise, 13.82 FPS and mAP50 0.537. ONNX FP32 failed to run a valid benchmark for aggregated model, but succeeded in inference of isolated instances. For the tested CPU-only Raspberry Pi 5 it is recommended to perform inference with the TFLite INT8 representation. It can achieve the lowest mean latency, the highest throughput and the lowest peak process memory among all completed formats, plus complete 90 image benchmark, and short continuous stability evidence. But it is not necessarily best always to use INT8. The appropriate runtime may be changed as a result of changes in accuracy requirements, hardware acceleration, cooling or field conditions.

5.2 Assessment of Project Objectives and Research Hypothesis

Table 5.1: Assessment of the project objectives against completed evidence.

Objective	Status
Establish the two-class detector and evaluation context	Achieved
Prepare six alternative edge-runtime representations	Achieved
Create a reproducible Raspberry Pi evaluation workflow	Achieved
Benchmark quality, latency, throughput and resource behaviour	Achieved for five formats; ONNX aggregate incomplete
Investigate operational reliability and negative results	Partially achieved; short/asymmetric stability evidence
Select and justify a preferred representation for the tested Pi	Achieved: TFLite INT8

There is support for the research hypothesis. Upon each successful deployment representation, the mean latency has been decreased in comparison with PyTorch FP32. The speed-up is roughly between $2.77\times$ for TFLite FP32 and $9.01\times$ for TFLite INT8. The wide range of enhancements has been seen over time. It has also been speculated that the accuracy or robustness penalty may occur: INT8 saw the greatest accuracy reduction, ONNX failed to run a valid continued benchmark, and NCNN FP32 failed

to pass the continuous attempts after completing the fixed benchmark. The evidence, then, supports the proposed accuracy-efficiency-robustness compromise, rather than a statement that lower precision is always better.

5.3 Current Project Status and Limitations

This work is a proof of an academic idea of an edge deployment system. It shows something that works with stored image rock detection and classification based on burial status on Raspberry Pi 5. This is not a complete autonomous field solution, though. However, live-camera acquisition, GPS integration, verified rock size/weight estimation and 3D sensing/robotic removal are not implemented in the larger project. The most important restriction in the model level is the half-buried recognition. The difference between fully-exposed and half-buried rocks is also significant and should not be solely due to conversion to INT8 or FP16. Other limitations of this experiment include a single Raspberry Pi, a single pass per format completed, fixed thresholds, and use of the 90 images as a workload of stored images from which to choose as a final runtime selection after completing a use as untouched architecture selection test. An unused deployment-validation set was not available. The second set of untouched ones is needed so a more accurate estimation of the final runtime can be done for deployment validation.

5.4 Prioritised Future Work

First of all, it is necessary to increase object recognition ability if the objects are partially buried in the ground. Independently collected additional samples should have various soils, lighting, moisture, occlusions, rock textures and dense scenes. The methods used to produce hard-negative mining and augmentation should be tested on a held-out field set and not based on training behavior. Secondly, several timed passes need to be made to test the actual runtime performance and longer stability testing under controlled ambient conditions would be expected. The cooling effect should be taken into account directly for the active cooling. The recording should include temperature, frequency/throttling state, power and latency. It is advisable to use the same continuous protocol as TFLite INT8 as NCNN FP16 maintains higher accuracy retention. Thirdly, the failure diagnosis must be carried out step by step for ONNX. The repeated runs should take into account the memory, powers, thermal and runtime parameters without assuming that the unexpected restart occurred because of these. ONNX should only be added to the total ranking after a controlled run. The fourth is that it's really crucial to shift from stored images to live-camera pipeline. The end-to-end measurements shall cover the acquisition, resizing/letterboxing, inference, post-processing and output handling. If this occurs, then the overall project can continue with the issues of GPS association, rock size estimation, multimodal or depth sensing and eventually integrate with robotic removal. Such procedures cannot be taken for granted because they are a new set of problems for validation; they are not expected to occur during successful inference of the detector.

5.5 Final Conclusion

A lightweight detector for agriculture has been tested on the usefulness rate of Raspberry Pi 5 for the project. The choice of deployments affects their accuracy, latency, resource usage and confidence in operation. TFLite INT8 provides the strongest overall evidence for the tested CPU-only proof of concept. NCNN FP16 offers higher accuracy retention than TFLite INT8, while with lower throughput. The most crucial challenge to be faced will not only be speeding up the inference process, but also reinforcing half-buried recognition and accepted sustained inference under cooled and live-camera condition in realistic field condition. The above contributions are a result of the group project. Input from individual roles, decisions and learning are recorded in the Individual Reflective Accounts.

Bibliography

- [1] Mar Ariza-Sentís, Sergio Vélez, Raquel Martínez-Peña, Hilmy Baja, and João Valente. Object detection and tracking in precision farming: a systematic review. *Computers and Electronics in Agriculture*, 219:108757, 2024.
- [2] Chetan M. Badgujar, Alwin Poulose, and Hao Gan. Agricultural object detection with you only look once (YOLO) algorithm: A bibliometric and systematic literature review. *Computers and Electronics in Agriculture*, 223:109090, 2024.
- [3] Felipe Campelo. On-the-go detection of field rocks for precision agriculture in potato fields: project kickoff slides, 2026. University of Bristol group-project meeting material, unpublished.
- [4] Dalhousie University and McCain Foods. On-the-go detection of field rocks for precision agriculture in potato fields: project flyer, 2026. Unpublished project material supplied to the authors.
- [5] Google. LiteRT documentation, 2026. Accessed 23 August 2026.
- [6] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2704–2713, 2018.
- [7] Astina Joice, Talha Tufaique, Humeera Tazeen, C. Igathinathane, Zhao Zhang, Craig Whippo, John Hendrickson, and David Archer. Applications of raspberry pi for precision agriculture—a systematic review. *Agriculture*, 15(3):227, 2025.
- [8] Yogeswaranathan Kalyani and Rem Collier. A systematic survey on the role of cloud, fog, and edge computing combination in smart agriculture. *Sensors*, 21(17):5922, 2021.
- [9] Xiang Li, Wenhai Wang, Lijun Wu, Shuo Chen, Xiaolin Hu, Jun Li, Jinhui Tang, and Jian Yang. Generalized focal loss: Learning qualified and distributed bounding boxes for dense object detection. In *Advances in Neural Information Processing Systems*, volume 33, pages 21002–21012, 2020.
- [10] Tsung-Yi Lin, Piotr Dollár, Ross Girshick, Kaiming He, Bharath Hariharan, and Serge Belongie. Feature pyramid networks for object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2117–2125, 2017.
- [11] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft COCO: Common objects in context. In *Computer Vision—ECCV 2014*, volume 8693 of *Lecture Notes in Computer Science*, pages 740–755. Springer, 2014.
- [12] Wei Liu, Dragomir Anguelov, Dumitru Erhan, Christian Szegedy, Scott Reed, Cheng-Yang Fu, and Alexander C. Berg. SSD: Single shot multibox detector. In *Computer Vision—ECCV 2016*, volume 9905 of *Lecture Notes in Computer Science*, pages 21–37. Springer, 2016.
- [13] Microsoft. ONNX Runtime documentation, 2026. Accessed 23 August 2026.
- [14] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 8748–8763, 2021.

- [15] Raspberry Pi Ltd. Cooling raspberry pi 5, 2026. Accessed 23 August 2026.
- [16] Raspberry Pi Ltd. Raspberry pi 5 documentation, 2026. Accessed 23 August 2026.
- [17] Vijay Janapa Reddi, Christine Cheng, David Kanter, Peter Mattson, Guenther Schmuelling, Carole-Jean Wu, Brian Anderson, Maximilien Breughe, Mark Charlebois, William Chou, et al. MLPerf inference benchmark. In *Proceedings of the ACM/IEEE 47th Annual International Symposium on Computer Architecture*, pages 446–459, 2020.
- [18] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 779–788, 2016.
- [19] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. In *Advances in Neural Information Processing Systems*, volume 28, pages 91–99, 2015.
- [20] Sapna, Himanshu Gauttam, Vikas Chauhan, Kiran Kumar Pattanaik, Aditya Trivedi, and Hrishita Ghosh. A comprehensive review of edge computing empowered smart agriculture: Trends, opportunities and future directions. *Computers and Electronics in Agriculture*, 241:111252, 2026.
- [21] Tencent. NCNN: a high-performance neural-network inference computing framework optimised for mobile platforms, 2026. Accessed 23 August 2026.
- [22] Florian Thürkow, Mike Teucher, Detlef Thürkow, and Milena Mohri. Stone detection on agricultural land using thermal imagery from unmanned aerial systems. *AgriEngineering*, 7(7):203, 2025.
- [23] Ultralytics. Ultralytics YOLO model documentation, 2026. Accessed 23 August 2026.
- [24] Jason Yosinski, Jeff Clune, Yoshua Bengio, and Hod Lipson. How transferable are features in deep neural networks? In *Advances in Neural Information Processing Systems*, volume 27, pages 3320–3328, 2014.

Appendix A

Dataset and Detector Development Evidence

There were originally 448 images and 706 labelled rocks: 362 fully exposed and 344 half buried. The pre-augmentation training/validation/test split is reported on Table 3.1. Training only augmentation produced 1,878 more images, yielding 3,563 occurrences of annotation and 2,191 training images in the copies with augmentation. The chosen model YOLOv8n achieved an untouched-test mAP50-95

Table A.1: Extended detector-development evidence.

Model	Precision	Recall	F1	mAP50	mAP50-95
YOLOv8n	0.681	0.759	0.718	0.759	0.4288
YOLO11n	0.743	0.719	0.731	0.729	0.4002
YOLO11s	0.764	0.801	0.782	0.783	0.4107

around 0.3384. At the evaluated source-detector operating point, the results of the confusion analysis represented 64/69 completely exposed rocks correctly and 23/66 half-buried rocks correctly. These values are operating-point correct rates and should not be changed to AP.

Appendix B

Raspberry Pi Deployment and Reproducibility

The six prepared representations were PyTorch FP32, ONNX FP32, TFLite FP32, TFLite INT8, NCNN FP32 and NCNN FP16. Inference was finally done with 640×640 letterboxed images with padding value 114. We use three warm-up inferences. A number of 90 images was taken in one timed pass. The ten image smoke test was a subset of the 90 images. It was used only as a functional check. The 200 original training images were used for INT8 calibration, with 304 labelled objects being balanced at 152 completely exposed and 152 half buried. The Raspberry Pi OS was Debian GNU/Linux 13. Python 3.13.5, Ultralytics 8.4.115, ONNX Runtime 1.28.0, OpenCV 5.0.0, NumPy 2.5.1, Pandas 3.0.5 and psutil 7.2.2 were used. The power supply was the official 27 W supply rated at 5.1 V/5 A and no Active Cooler was employed. The benchmark was conducted without the use of a camera.

Appendix C

Extended Raspberry Pi Results

Table C.1: Extended canonical edge metrics which are used in the main evaluation.

Format	Latency ms	FPS	CPU %	RAM MB	P	R	F1	mAP50	mAP50-95	Count MAE
PyTorch FP32	397.48	2.52	44.47	839.47	0.716	0.550	0.622	0.548	0.275	0.600
TFLite FP32	143.70	6.95	69.95	868.27	0.669	0.542	0.599	0.535	0.270	0.600
TFLite INT8	44.13	22.62	62.83	824.22	0.661	0.539	0.594	0.503	0.234	0.633
NCNN FP32	75.09	13.26	83.32	835.25	0.667	0.542	0.598	0.535	0.271	0.600
NCNN FP16	72.31	13.82	87.08	835.45	0.672	0.542	0.600	0.537	0.270	0.611
ONNX FP32	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A

The approximate maximum temperatures at the fixed benchmarks were 55.10, 56.75, 59.50, 61.70 and 66.10 °C for TFLite INT8, NCNN FP32, NCNN FP16, PyTorch FP32 and TFLite FP32 respectively. These are observations of short workloads and not long-run thermal equilibria.

Appendix D

Stability and Failure Diagnostics

The only valid continuous-inference experiment kept for final analysis was for TFLite INT8. It took approximately 2 minutes, approximately 2,533 inference rows, and approximately 22.7 FPS. Temperature rose from the mid-40s to about 76–77 °C, with a raw recorded maximum of approximately 77.1 °C. No rebooting or throttling was recorded while running. ONNX FP32 could process isolated images, but did not perform a valid 90-image aggregate benchmark; sustained attempts caused abrupt Raspberry Pi reboots, and the cause of the issue is still unknown. NCNN FP32 managed to complete the 90-image benchmark test, but a follow-up continuous attempt hard reset the device and did not produce a valid completed continuous CSV. No genuine 30-minute or 10-minute final stability test was completed. A file named as a 30-minute test duplicated the genuine 2-minute data and was therefore not included in the evidence base.